Multilingual Entity Insertion in Wikipedia

Thesis submitted on 19 January 2024
to the School of Basic Sciences
regarding work done at the Data Science Laboratory, School of Computer and Communication Sciences
for completion of the Master's program in Computational Science and Engineering
École Polytechnique Fédérale de Lausanne

Tomás Feith

Akhil Arora, Thesis Advisor
Professor Robert West, Thesis Supervisor





Acknowledgements

The people we meet and the interactions we have with them help shape who we are, how we think, and what we do. I know I would not have come to where I am now if I had not had so many people support me and push me to be better, to be all that I can. In that sense, this work, as well as all the work that brought me here, is a massive tapestry of all those people who helped me grow. To all of them, I want to say, thank you.

I would like to thank Akhil Arora for the many discussions and feedback sessions that allowed the thesis to grow in the form it has today. I would also like to thank Robert West for making me feel welcome at DLAB and for giving me my first guided tour of the world of data analysis and computational linguistics.

There have been several teachers and mentors who completely drove the focus of my academic career. Thank you Madalena Lambéria, for showing me that there was so much more to learn outside of high school and taking me to my first steps in the world of Physics, even if I did end up not staying there. Thank you Ermelinda Mineiro, for teaching me so much about perseverance, and discipline, and always pushing me to work harder. Thank you to Pedro Sebastião, Pedro Assis, Vânia Silvério, who taught me to think critically and how to handle uncertainty and errors in my data. And thank you to Arno Schneuwly, who taught me so much about industry research, large-scale projects and the corporate world when I was just a student fresh out of university who didn't really know anything.

To all my friends, who have made this long adventure so much more lighter. To Miguel Roldão, Miguel Farias and André Charneca, who will always be my favourite lab partners and with whom I had so much fun playing with sand, magnets, and pendulums. To Jean Alves who was always one of my top picks to work with because she made the work feel much less like work. To Maria Casimiro and Sara Cardoso, who were there in very different times of my life but both helped me learn a lot about myself and who I wanted to be. To David Pujol, who was always down for a climbing session or a hike up the mountain. To Gabriele Furlan, who showed me the wonders of guanciale and proper Italian cuisine. And to Nawar, to Francesco, to Zeno, to Elena, to Jorge for all the happy hours and the fun times we had in Lausanne.

I want to give my biggest thank you to my parents. You have always made sure I had everything I needed to thrive, and even when things were tough you shielded me and allowed me to keep growing into the person I am today. All the late practice pick-ups, all the drives back and forth

from Lisbon, all the trips that showed me there was a bigger world out there to explore, and all the food, movies, games, museums, hikes. Thank you for always letting me be me, and always giving me the space and encouragement to figure out this thing called life.

Last but not least, I want to thank Rosarinho, my love. Thank you for being my rock, my support, my cheerleader. You have made the last two years of my life so much better, and this wouldn't have been possible without you.

Lausanne, January 19, 2024 Tomás Feith

Abstract

Links are a fundamental part of information networks. Links are what turn isolated pieces of knowledge into a flowing network of information that is much richer than the sum of its parts, by facilitating navigation, connecting related entities, and bridging different concepts. However, the task of inserting a link into an article is not trivial, as it requires understanding both the source and target entities and reading the entire article to find the best text span in which to insert the link. In this thesis, our goal is to study link insertion in Wikipedia. For this purpose, we introduce a novel task called entity insertion. We empirically find that this problem is important and challenging for humans, as in more than 55% of the cases we find that none of the existing text is suitable to insert the entity, and new text has to be inserted along with the inserted entity by a human editor. To facilitate further entity insertion research, we produce a dataset of 110 languages with links extracted from Wikipedia. The dataset consists of all the links available at the timestamp of 2023-10-01, as well as the links added in the period between 2023-10-01 and 2023-11-01. We additionally develop a multilingual entity insertion model trained and evaluated on a subset of 21 languages. Our results show that when the text to be hyperlinked is missing, strong classical baselines behave similarly to random and are unable to identify the correct candidates, justifying the need for complex models. We found that multilingual models can transfer knowledge from multiple languages to improve performance beyond what models fine-tuned on monolingual data can achieve. We further show that multilingual models can be applied in a zero-shot manner on languages they were not fine-tuned on with minimal performance drop, which is vital for the large-scale operationalization of entity insertion models in Wikipedia. These results are a starting point for entity insertion, and we hope that the dataset we release will lead to further advances in multilingual models and their applications for entity insertion.

Contents

Acknowledgements **i**

Abstract **iii**

List of Figures **vii**

List of Tables **ix**

Introduction **1**

1 Entity Insertion **5**
1.1 Task definition 5
1.2 Potential use case 6
1.3 Beyond Wikipedia 7

2 Background and Related Work **9**
2.1 Entity insertion: related work 9
2.1.1 Entity linking 9
2.1.2 Other related tasks 10
2.1.3 Key differences 11
2.2 Modelling background 11
2.2.1 Pre-trained language models 11
2.2.2 Ranking tasks 12
2.2.3 Domain adaption 12

3 Dataset Construction **15**
3.1 Data processing 15
3.1.1 Existing links 15
3.1.2 Added links 16
3.2 Two-stage dataset 18

4 Modelling Framework **21**
4.1 Model architecture 21
4.2 Training 22
4.3 Introduced novelties 23

4.3.1 Data augmentation . . . 23
4.3.2 Knowledge injection . . . 24
4.3.3 Two-stage training pipeline . . . 25

5 Experiments **29**
5.1 Data . . . 29
5.1.1 Negative candidates . . . 30
5.2 Baselines . . . 30
5.2.1 String match . . . 30
5.2.2 BM25 . . . 31
5.3 Multilingual entity insertion . . . 32
5.3.1 Zero-shot vs fine-tuned . . . 33
5.4 Ablations . . . 35
5.4.1 Starting model comparison . . . 35
5.4.2 Model size comparison . . . 36
5.4.3 Training size . . . 37
5.4.4 Impact of training stages . . . 38
5.4.5 Impact of introduced novelties . . . 40
5.4.6 Multilingual model vs monolingual model . . . 42

Conclusions and Future Work **45**
5.5 Conclusions . . . 45
5.6 Future work . . . 45

Bibliography **52**

A Full Link Insertion Statistics **53**

B Single Encoder vs Triple Encoder **55**

List of Figures

1 Distribution of link insertion strategies in Wikipedia, for 21 languages 3

1.1 Examples of entity insertion in the form of link insertion in Wikipedia 6

4.1 Difference between the base modelling framework and the improved modelling framework 23
4.2 Training loss for a two-stage training run 26

5.1 Multilingual results 33
5.2 Comparison of zero-shot performance with monolingual and multilingual performance 35
5.3 Comparison of the impact of the amount of data on model performance for the two training stages 38

B.1 Discarded architecture diagram 56

List of Tables

1 Pages and links statistics for 21 Wikipedia languages 2

3.1 Statistics about the number of articles and links removed during post-processing 17
3.2 Examples of different link insertion strategies 19
3.3 Comparison between the number of links for stage 1 and stage 2 20

4.1 Examples of different strategies for dynamic context masking 27

5.1 Aggregate multilingual results 34
5.2 Relative average performance of the models in Figure 5.2 34
5.3 Comparison between the two multilingual model variants, M_{LARGE} and M_{SMALL} 36
5.4 Relevant statistics for different models considered 36
5.5 Entity insertion performance for different starting models 37
5.6 Entity insertion performance for different model sizes 37
5.7 Impact of the different training stages for the monolingual models 40
5.8 Impact of the different training stages for the multilingual model 41
5.9 Impact of the introduced novelties on performance 42
5.10 Entity insertion performance for a monolingual model and a multilingual model 43

A.1 Full statistics for added links for all 110 processed languages 54
A.2 Aggregated statistics for added links for all 110 processed languages 54

Introduction

The existence of links in information networks is of paramount importance. Links between distinct entities allow information to flow through the network, by enabling navigability and connecting related concepts. Content-rich networks are particularly reliant on links, as they require a dense network of connections for the full potential of the knowledge contained in them to be exploited. Wikipedia, with over 62 million articles in around 300 languages, is one of the largest content-rich networks ever created. With such a large number of articles, links are vital to prevent pages from becoming orphaned[1], dead-ends[2], or walled gardens[3], all of which diminish the usability of Wikipedia.

Nevertheless, the insertion of links in Wikipedia is not a trivial process. Not only does it require finding source-target pairs worth linking, but once the pair is found the editor needs to understand both the source and the target entities and read the entire source article to determine where to insert the link. This places a large cognitive load on the editors, requiring them to spend significant effort to insert links. This added cognitive load has an even larger impact in smaller languages, which have fewer editors. Table 1 shows that, in general, languages with fewer editors not only have fewer pages but also have sparser pages, with fewer links per page. Sparser pages have reduced usability, making it harder to navigate to related articles and connect different concepts. For this reason, it is vital to develop new tools to facilitate the task of link insertion and operationalise link recommendation in Wikipedia and other information networks, decreasing the sparsity of lower resource languages.

There is previous research (Gerlach et al., 2021) on tools to operationalise link recommendation in Wikipedia, and there are several tools already implemented, such as AddLink [4]. However, the developed tools focus only on the task of link prediction, i.e., the task of predicting a target entity given a source entity. For this reason, we choose to tackle the complementary problem of link insertion, i.e., the task of predicting where on the page to insert a link to a target entity.

Given a source page and a link to be inserted, there are two ways to add the link to the page. One way is to hyperlink a part of the text already present on the page. The second way is to

[1] https://en.wikipedia.org/wiki/Wikipedia:Orphan
[2] https://en.wikipedia.org/wiki/Wikipedia:Dead-end_pages
[3] https://en.wikipedia.org/wiki/Wikipedia:Walled_garden
[4] https://wikitech.wikimedia.org/wiki/Add_Link

Table 1: **Pages and links statistics for 21 Wikipedia languages.** Statistics extracted from the 2023-11-01 Wikipedia timestamp.

	Language	Active Editors	#Pages	#Links	#Links / #Pages
en	English	46.7M	6.7M	170.0M	25.4
fr	French	4.8M	2.6M	94.0M	36.2
pt	Portuguese	3.0M	1.1M	22.8M	20.7
it	Italian	2.5M	1.8M	57.0M	31.7
ja	Japanese	2.2M	1.4M	62.7M	44.8
simple	Simple English	1.4M	242K	2.8M	11.6
hi	Hindi	795K	163K	1.3M	8.0
cs	Czech	645K	534K	15.0M	28.1
ms	Malay	327K	369K	3.1M	8.4
sk	Slovak	244K	246K	4.1M	16.7
ur	Urdu	173K	200K	1.6M	8.0
af	Afrikaans	171K	112K	1.4M	12.5
kk	Kazakh	146K	239K	1.7M	7.1
uz	Uzbek	118K	248K	1.5M	6.0
is	Icelandic	96K	57K	783K	13.7
cy	Welsh	86K	280K	3.0M	10.7
kn	Kannada	85K	31K	301K	9.7
gu	Gujarati	76K	30K	415K	13.8
sw	Swahili	64K	79K	1.1M	13.9
ga	Irish	59K	59K	469K	7.9
ps	Pashto	32K	21K	71K	3.4

add text to the page and hyperlink part of the added text. Previous work has, to the best of our knowledge, exclusively focused on the first scenario by framing the task as an entity-linking problem, in which the goal is to assign a unique identity to entities mentioned in the text Gerlach et al. (2021); Mihalcea and Csomai (2007); Milne and Witten (2008); West et al. (2009). However, previous approaches have a key underlying assumption; the mention is present in the text at inference time. For the second scenario, this assumption does not hold, as the editors simultaneously add the text and hyperlink it. As such, since the mention is not present at edit time, none of the previous approaches are applicable.

We found that the majority of links added to Wikipedia fall into the second scenario, as shown in Figure 1. For a selection of 21 languages, only 28% of the added links were inserted in the already existing text. For the remaining links, the editors added the mention to an already existing sentence (20%), added a new sentence (18%), added a span of multiple sentences (24%), or added a new section (10%), and then hyperlinked part of the added text. These results show that there is a high-impact need for a tool that can handle both link insertion scenarios. We introduce a novel task titled entity insertion, and we show how a model trained for entity insertion is capable of predicting where to insert the entity whether the mentions are present or not.

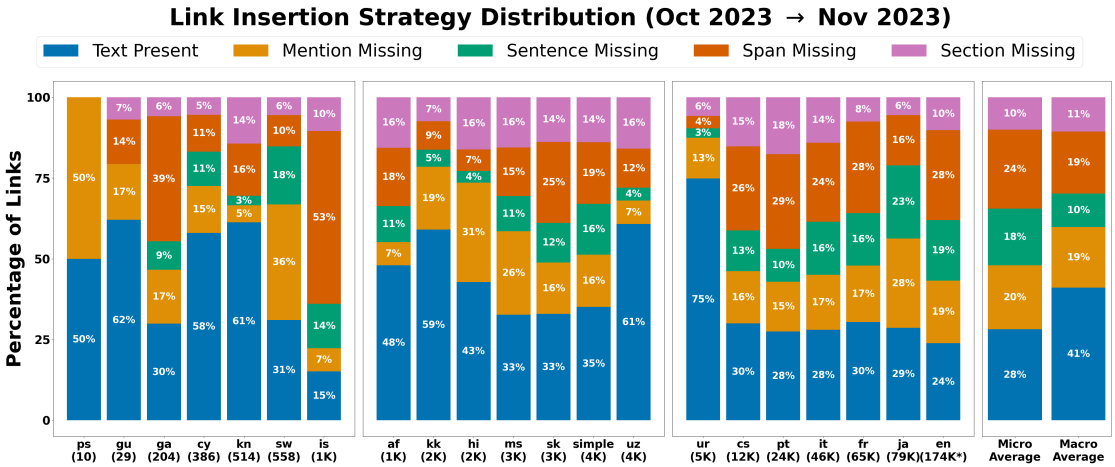


Figure 1: **Distribution of link insertion strategies in Wikipedia, for 21 languages.** Horizontal axis shows language code and number of added links. Due to time and computational restraints, only a sample of the links was extracted for English.

Summary of contributions

Our contributions are as follows:

- We introduce the novel task of entity insertion in Wikipedia (Chapter 1);
- We make available a dataset of 110 languages containing links from Wikipedia articles to enable further research into entity insertion (Chapter 3);
- We introduce a novel data augmentation strategy, as well as a two-stage training pipeline for data domain adaptation (Chapter 4);
- We study the zero-shot capabilities and the cross-lingual knowledge transfer capabilities of multilingual models (Chapter 5);
- We suggest multiple directions for future work on multilingual entity insertion.

1 Entity Insertion

This chapter aims to present a rigorous introduction to the entity insertion task. To achieve this, we first define the task. Then, we present a potential use case where an entity insertion module may thrive and previous approaches would fail, and discuss applications of entity insertion modules beyond link insertion in Wikipedia.

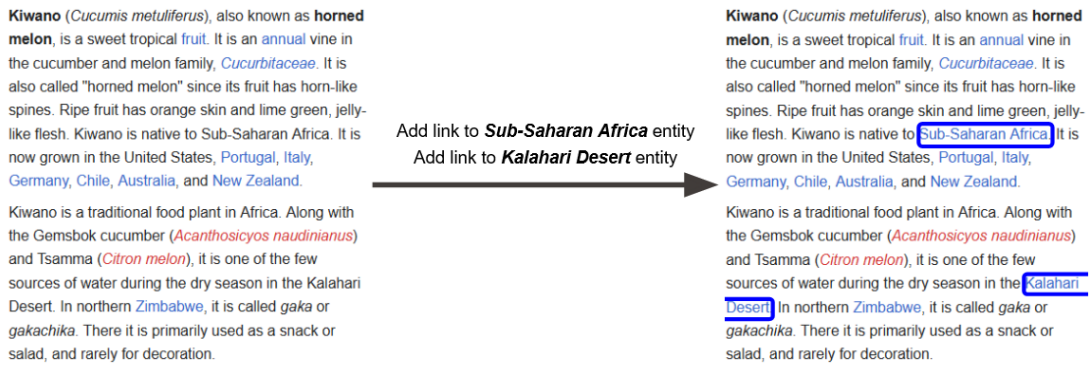
1.1 Task definition

Let E_{src} be a source entity and E_{tgt} be a target entity. Let X_{src} be the textual content of the article corresponding to the entity E_{src}. The text can be partitioned into a set of (potentially overlapping) text spans, $\mathscr{X}_{src} = \{x_1, ..., x_M\}$, where M is the number of text spans selected. Entity insertion is the task of selecting the span x^* in which entity E_{tgt} is most relevant

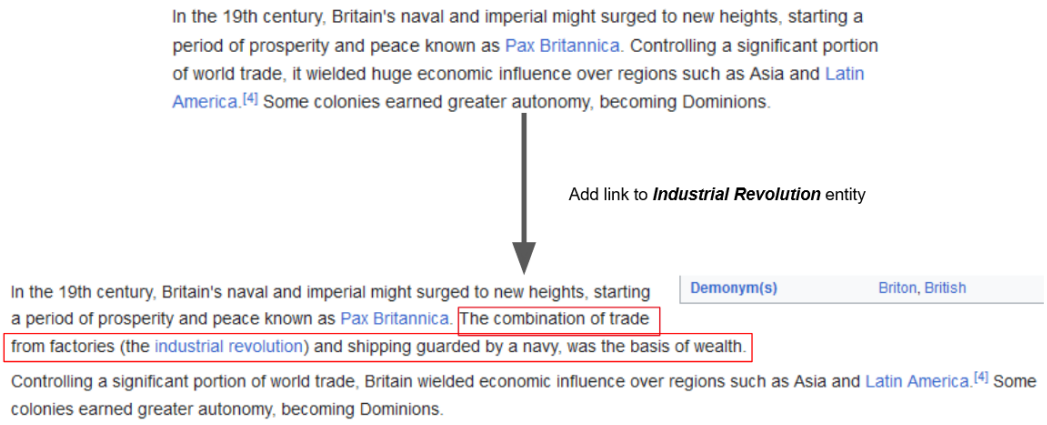
$$x^* = \arg\max_{x \in \mathscr{X}_{src}} \mathscr{R}(x, E_{tgt}) \tag{1.1}$$

where $\mathscr{R}$ is an arbitrary relevance function quantifying the relevance of E_{tgt} to each text span $x \in \mathscr{X}_{src}$. Figure 1.1 shows two examples of entity insertion in the context of link insertion. In both scenarios, an entity model should select the best text span where to add a link to the target entities needed. As seen in Figure 1.1b, the mention linking to the target entity does not yet need to be in the source article, in which case the model should exploit context information to determine the best text span in which to insert the link to the entity.

In our work, we frame the novel task of entity insertion as a ranking problem. Given a source entity E_{src}, a target entity E_{tgt}, and the textual content X_{src} of the source article, the goal of entity insertion is to find where to insert E_{tgt} in X_{src}. This task can thus be solved by ranking all the candidate text spans $\mathscr{X}_{src}$ by how related the target entity is to the text span. Given this framework, we propose a strong entity insertion model, described in Chapter 4.



(a) **Easy entity insertion example.** The editor hyperlinks already existing text.



(b) **Hard entity insertion example.** The editor adds text to the article and simultaneously hyperlinks part of the added text.

Figure 1.1: **Examples of entity insertion in the form of link insertion in Wikipedia.**

1.2 Potential use case

In this section, we present the use case of Wikipedia page de-orphaning. Orphan pages are Wikipedia pages with no inbound links, i.e., no other page in Wikipedia contains a link to the orphan page. While these pages can still be accessed by searching Wikipedia, orphan pages are difficult to find, which means these pages receive less readership and improvement from the community.

Previous work (Arora et al., 2023) in page de-orphaning exploited the multilingual knowledge present in Wikipedia and transferred links from one language to another. Let E_1 be an entity associated with article $A_1^{l_1}$, which is an orphan article in language l_1. There may exist a language l_2 where $A_1^{l_2}$ is not an orphan, and can be accessed via article $A_2^{l_2}$. Then, we can try to insert a link to $A_1^{l_1}$ in $A_2^{l_1}$ because even though the articles are in different languages, they are a representation of the same entities, and if they are connected in one language then they may also be connected in another.

Considering the scenario described above, an entity insertion module is much better suited for page de-orphaning than an entity linking module because the latter relies on the existence of the mention in the article to insert the link. On the other hand, an entity insertion module can find the text span most related to the orphan page, regardless of whether the orphan page is mentioned or not. Since orphan pages are often related to obscure and/or recent entities, arguably a significant number of links will need to be added at the same time as the mention text to be linked, meaning that an entity insertion module is key for effective page de-orphaning.

This example additionally shows that there are naturally occurring use cases where both the source and the target entities are known, and the goal is to learn where to insert the link, further justifying the introduction of the novel task of entity insertion.

1.3 Beyond Wikipedia

In our work, we focus on the entity insertion task in the scope of Wikipedia, under the form of link insertion. Nevertheless, our framework is general and can be used in several applications beyond Wikipedia.

One such application is citation insertion. The goal of citation insertion is, given a source paper and a target paper to be cited, to find where in the source paper to include the citation for the target paper. In this task, the papers are the entities to link and the text spans are extracted from the source paper. Just like in the link insertion application, there has been little work in citation insertion. Previous work (Taylor et al., 2022; Jiang et al., 2019; Bhagavatula et al., 2018) mainly focused on citation recommendation, the analogous task of entity linking in which the goal is to find relevant target papers to cite given a source paper. Nevertheless, citation insertion is a valid and relevant use case, as automatic citation insertion, possibly allied with citation recommendation, could significantly reduce the cognitive load of literature review tasks.

2 Background and Related Work

2.1 Entity insertion: related work

2.1.1 Entity linking

Previous work in link insertion has framed link insertion as an entity linking problem (Gerlach et al., 2021; Mihalcea and Csomai, 2007; Milne and Witten, 2008; West et al., 2009; Arora et al., 2021; Čuljak et al., 2022). The goal of entity linking is to assign a unique identity to entities mentioned in the text.

Let E_{src} be a source entity and x_{src} be a text span in the article associated with E_{src}. Let $\mathscr{E}$ be the space of all entities in a given domain (e.g., Wikipedia). The task of entity linking is composed of two sub-tasks: Named Entity Recognition (NER) and Named Entity Disambiguation (NED). The problems can be formally defined as

$$\text{(NER): } l^*, r^* = \arg \max_{\substack{(l,r)\in([1,L]\times[1,L]) \\ l<r}} \mathscr{R}_{NER}(x_{src}, l, r, \left[E^*_{tgt}\right]) \tag{2.1}$$

$$\text{(NED): } E^*_{tgt} = \arg\max_{e\in\mathscr{E}} \mathscr{R}_{NED}(x_{src}, e, \left[l^*\right], \left[r^*\right]) \tag{2.2}$$

where E^*_{tgt} is the predicted target entity, l^* and r^* are the left and right demarcations of the predicted mention in x_{src}, L is the length of x_{src} and $\mathscr{R}_{NED}$ and $\mathscr{R}_{NER}$ are arbitrary relevance functions quantifying the relevance of the target entities and the mention demarcations, respectively, to the text span. Arguments in $[\cdot]$ are optional because equations 2.1 and 2.2 show a circular dependency, which means one of the problems needs to be solved first with only partial information. Most research (Hoffart et al., 2011; Fu et al., 2020; van Hulst et al., 2020) into entity linking solves first the NER problem with no knowledge of E^*_{tgt}. In this approach, NER is the task of finding any candidate mentions for named entities in the source article. However, there is recent work (Zhang et al., 2022) exploring the problem in reverse order, first solving NED by finding target entities related to the source article and then NER searching only for mentions for the found target E^*_{tgt}.

When the mention is present, the task of entity insertion is similar to NER as framed by Zhang et al. (2022), in which the target entity is known, as both tasks can be solved by searching for mentions in the text. However, entity insertion is a more general task as it allows for the mention to the target entity to not yet be in the text, in which case the goal is to exploit the context information to find the text span most related to the target entity. NER modules (Nothman et al., 2013; Lin and Wu, 2009; Finkel et al., 2005) are designed to search for the most related mentions, and as such they are not applicable in scenarios where the mentions are not yet available.

NED, on the other hand, can be considered to be the dual task of entity insertion. While both tasks attempt to optimise the relevance of the target entity to the text span, the optimisation domains are different. Comparing Equations 1.1 and 2.2 we see that in entity insertion the target entity is fixed, and the relevance is maximised by searching in the space $\mathscr{X}_{src}$, the set of text spans in the article X_{src}. NED maximises the target relevance by searching in $\mathscr{E}$, which is the space of all entities in the target domain. As such, while the two tasks share a similar mathematical formalism, they have a different semantic interpretation and a different set of downstream applications.

2.1.2 Other related tasks

Entity tagging, as introduced by Du et al. (2022), is a relaxed form of entity linking. Under the entity tagging framework, the model is only tasked with determining the entities present in the text and does not need to find the exact mentions related to the entity. However, even though the model is not tasked with extracting the target entity's mention, the task of entity tagging still assumes that the text contains some mention to the target entity, which distinguishes it from entity insertion.

Link the Wiki (Huang et al., 2008) was a track at INEX 2008 which offered two tasks: file-to-file link discovery and mention-to-BEP (best entry point) link discovery[1]. File-to-file link discovery is a document-level task that can be framed as a graph edge prediction task, where the Wikipedia articles act as nodes and the links act as edges. The mention-to-BEP task is an entity linking task with anchor prediction, where the two-part goal is to find mentions in the source article pointing to other articles, and finding the best point of entry (the anchor) in the target file. The mention-to-BEP task has more recently resurfaced as an anchor prediction task (Liu et al., 2023). Nevertheless, the authors found that human annotators considered more than 98% of the links extracted to be unanchorable. We believe this reflects the usage of links on the web, making anchor prediction an unnatural task, and that there is vastly more gain to be extracted from the task of entity insertion.

[1] In the original paper, the authors call the second task *anchor-to-BEP* because they define anchors as the text hyperlinked to another entity. We call it mention-to-BEP to be consistent with the notation used in our work.

2.1.3 Key differences

Entity insertion is different from all the presented tasks in key ways.

1. **Text Presence**: All the tasks presented above have one underlying assumption: the text to be hyperlinked to the target entity is already present at inference time. Entity insertion removes that assumption and instead allows the context of each text span to also contribute to the relevance score between the text span and the target entity. As such, entity insertion is capable of producing link insertion recommendations even when the mention is not present, which none of the presented approaches are capable of doing.

2. **Optimisation Domain**: NED and entity tagging are formally similar to entity insertion, as all three tasks attempt to maximise the relevance metric between a text span and a target entity. However, NED and entity tagging attempt to solve the problem by finding the most relevant target entity, by searching in the entity space. On the other hand, entity insertion attempts to solve the optimisation problem by finding the text span most related to the target entity, by searching in the text space.

3. **Optimising Source Text vs Optimising Target Text**: Mention-to-BEP and anchor prediction perform the reverse task of entity insertion, by trying to find the best text span in the target text, not in the source text. However, finding the best text span in the target text is unnatural on the web, as has been proven that humans find the vast majority of links to be unanchorable. On the other hand, in more than 55% of the links added to Wikipedia, we found that none of the existing text spans are suitable to insert the entity, and a new span of text has to be inserted along with the inserted entity by a human editor. This shows that entity insertion not only is a more natural task but also has a much higher potential benefit.

Given the differences presented above, we believe entity insertion to be a novel and worthwhile task, introducing new downstream applications that can benefit tasks such as link insertion on the web.

2.2 Modelling background

2.2.1 Pre-trained language models

The transformer architecture, introduced by Vaswani et al. (2017), has become the *de facto* architecture for most Natural Language Processing (NLP) applications. A transformer-based pre-trained language model takes as input a text sequence and computes a vector embedding that captures the semantic and structural information contained in the text sequence, which can then be used in downstream applications.

Pre-training is an expensive process. For example, the base variant of BERT (Devlin et al., 2019) took four days to train with 16 TPUs and RoBERTa (Liu et al., 2019) took one day to train with 1024 GPUs. However, pre-trained models can be leveraged to novel downstream tasks by fine-tuning them on task-specific datasets. As a comparison, the authors of BERT (Devlin et al., 2019) introduced several fine-tuned variants of BERT, all of which were fine-tuned in one hour using one TPU, which is much cheaper than pre-training the model for each task. This paradigm of pre-training language models on large amounts of data and then fine-tuning on much smaller amounts can reduce the cost of model training while retaining the knowledge from the pre-trained model and transferring it to the downstream task. Popular pre-trained models for multilingual tasks are mBERT (Devlin et al., 2019), XLM-RoBERTa (Conneau et al., 2020), and mT5 (Xue et al., 2021).

2.2.2 Ranking tasks

Classical approaches for ranking tasks, such as BM25 (Robertson and Zaragoza, 2009), mainly rely on probabilistic methods that attempt to match keywords between a query and a candidate document. However, these methods cannot capture complex semantic and structural patterns. For example, the sentences "The hero defeated the dragon and saved the damsel" and "The knight slayed the beast and rescued the princess" are semantically equivalent, but classical methods would to match them due to the small vocabulary overlap.

Because of this, pre-trained language models have become state-of-the-art for text ranking (Lin et al., 2021). A popular design for transformer-based ranking tasks is the cross-attention model, in which the query and the candidate document are concatenated into a sequence and then processed by the model. Since transformer models employ attention mechanisms, this strategy allows the model to capture the interactions between the query and the document.

This approach has been explored for encoder-only models (Han et al., 2020; Nogueira et al., 2019; Gao et al., 2021), outperforming classical methods. There has also been previous research (Nogueira et al., 2020; Ju et al., 2021) in exploring encoder-decoder models, such as T5 (Raffel et al., 2020). However, even though encoder-decoder models are typically larger than encoder-only models, RankT5 (Zhuang et al., 2023) has shown that there is no consistent winner between encoder-decoder and encoder-only models.

2.2.3 Domain adaption

Gururangan et al. (2020) have shown that a second phase of pre-training using domain-specific knowledge can improve the performance of language models. Their experiments started with a pre-trained RoBERTa model and continued pre-training it using unlabelled data from a large corpus of domain-specific text.

In our work, we propose a similar approach for fine-tuning, where we apply a first stage of domain-shifted data and then a second stage of domain-specific data to improve the

performance further.

3 Dataset Construction

We constructed a new dataset to study the entity insertion problem by using articles from Wikipedia, from 110 different languages. The dataset consists of all the links from all the Wikipedia pages and each link's surrounding context and additional meta-data (such as source title, target title, source section, source lead, and target lead). In total, we have constructed a dataset with 957.6M links, from 49.0M pages in 110 languages. The biggest language is English (en), with 166.7M links from 6.7M pages, and the smallest language is Xhosa (xh), with 2.8K links from 1.6K pages.

In this chapter, we'll detail the data processing pipeline.

3.1 Data processing

We processed the raw data in two ways. We first extracted all the links present at the timestamp of 2023-10-01. This gave us a static dataset of all the existing links, as well as the context around each link. Then we found all the links added in the time between 2023-10-01 and 2023-11-01. This second version of the data gave us not just the links, but information about the context existing in the page before the link was added (which may or may not have contained the mention to be linked).

3.1.1 Existing links

To find all the existing links for a given timestamp, we used the data available from the Wikimedia dumps[1]. We first extracted the HTML files for all the Wikipedia pages, as well as relevant meta-data (ID, Wikimedia QID, and whether the page is a redirect page). Then we processed all the HTML files to extract the links and their surrounding context. We only considered links from the main article body, ignoring all links in figures, tables, notes, and captions. This choice was driven by the fact that these elements have a strong structural

[1] https://dumps.wikimedia.org/

component which is hard to capture with language models, and thus they fall outside the scope of our work. Additionally, the vast majority of the links are present in the article body, making the article body the region with the highest potential benefit. Finally, when storing the links, we resolved all the redirects to their final target page

Once we had the links extracted, we applied the following post-processing steps. We removed all the articles without a lead paragraph, as we use the lead paragraph to compute the representation of the articles, and as such our method is not compatible with articles without a lead paragraph. Additionally, we found several errors in the Wikimedia dumps, and we removed all the articles where the HTML files were missing and all the articles where the QIDs were missing. The choice to remove the articles without HTML files is evident, as without the HTML files we cannot compute the article's representation nor can we extract the links in the page. We chose to remove the articles without QIDs because the QIDs allow us to link entities from different languages. For multilingual research, it may be relevant to explore complementary information in different languages, so we only kept the articles that allow us to do that. Regarding the links, we removed all the links where either the source or the target article was one of the removed articles. We also dropped all the self-links.

Table 3.1 shows statistics about the amount of data lost during post-processing. In total, 1.8% of the articles and 5.5% of the links were removed. However, we see that there is a high variance in the amount of data removed, with the number of pages removed varying from 0.7% to 23.1% and the number of links removed varying from 1.9% to 31.0%. The table shows that smaller languages are more affected by the post-processing pipeline (the 5-smallest languages had 9.4% of their articles removed, as opposed to 1.0% for the 5-largest languages). However, these results are to be expected, as smaller languages are more prone to having incomplete articles being removed by the post-processing pipeline. This then reflects on the removed links as well, leading to the higher percentages observed. Nevertheless, the results show that a vast majority of the data is of high quality and is kept for training and inference.

3.1.2 Added links

To find the links added between two consecutive months, we had to use two dumps, one for each month. First, we had to find all the links existing in each month, using the process described in Section 3.1.1. Then, we took the difference between the sets of links, to find the links existing in the second month but missing from the first month. This way, we knew which links were added between the two months.

However, we would like to know the exact changes the editors made when inserting the links. For this, we needed to process the revision history to find the revision ID of the article immediately before the link was added, and the revision ID of the article immediately after. The revision history is also available as part of the Wikimedia dumps, but only as XML files. For this reason, we first parsed the XML files to find the revision ID pairs, and then we downloaded the HTML pages directly from Wikipedia. Finally, once we had the two consecutive HTML

Table 3.1: **Statistics about the number of articles and links removed during post-processing.**

	All Articles	Clean Articles	Lost Articles	All Links	Clean Links	Lost Links
ps	20.6K	15.8K	4.7K (23.1%)	71.0K	49.0K	22.0K (31.0%)
gu	30.4K	30.2K	0.2K (0.8%)	415.0K	411.4K	3.6K (0.9%)
kn	31.3K	27.5K	3.8K (12.2%)	298.1K	249.5K	48.6K (16.3%)
is	57.3K	50.9K	6.4K (11.2%)	781.6K	612.2K	169.4K (21.7%)
ga	59.1K	55.6K	3.5K (5.9%)	467.5K	384.8K	82.7K (17.7%)
sw	78.5K	73.4K	5.1K (6.5%)	1.08M	1.01M	0.07M (6.7%)
af	111.6K	110.8K	0.8K (0.7%)	1.41M	1.39M	0.03M (1.9%)
hi	163.0K	154.7K	8.3K (5.1%)	1.26M	1.20M	0.06M (4.8%)
ur	198.6K	187.7K	10.8K (5.5%)	1.63M	1.45M	0.18M (11.3%)
kk	238.7K	232.3K	6.4K (2.7%)	1.71M	1.62M	0.09M (5.5%)
simple	240.2K	238.0K	2.3K (0.9%)	2.75M	2.59M	0.16M (5.6%)
sk	246.2K	240.5K	5.7K (2.3%)	4.07M	3.39M	0.68M (16.6%)
uz	246.6K	224.4K	22.2K (9.0%)	1.53M	1.33M	0.20M (13.3%)
cy	279.5K	269.3K	10.2K (3.6%)	3.05M	2.68M	0.37M (12.0%)
ms	368.2K	362.2K	6.0K (1.6%)	3.07M	2.92M	0.15M (5.0%)
cs	532.7K	526.2K	6.4K (1.2%)	14.99M	14.31M	0.68M (4.6%)
pt	1.11M	1.10M	0.01M (0.9%)	22.75M	21.89M	0.86M (3.8%)
ja	1.39M	1.37M	0.02M (1.4%)	62.57M	60.61M	1.96M (3.1%)
it	1.83M	1.77M	0.06M (3.5%)	56.81M	50.92M	5.89M (10.4%)
fr	2.56M	2.50M	0.06M (2.2%)	94.08M	84.67M	9.41M (10.0%)
en	6.72M	6.68M	0.05M (0.7%)	169.58M	166.06M	3.52M (2.1%)
Total	16.52M	16.21M	0.30M (1.8%)	444.41M	419.76M	24.65M (5.6%)

versions, we took the difference between the two to find which changes the editor made, thus finding how the links were added. We categorised the added links into five categories.

The first category is `text_present`. Links that fall into this category were added by hyperlinking an existing mention. They were detected by matching a sentence with a new link in the second version with a sentence in the first version, and checking that the link mention already existed in the first version.

The second category is `missing_mention`. These links were added by taking an already existing sentence and adding the mention for a new link (and potentially some additional context to the sentence). To detect such links, we found sentences in the first version that were similar to sentences in the second version containing new links. We defined similarity via word frequency cosine similarity, and applied a threshold of 0.5 to match the sentences in the two versions[2].

The third category is `missing_sentence`. Such links were added by writing a new sentence to complement the already existing text and hyperlinking part of the sentence at the same time. To detect these links, we found sentences with new links in the second version that could not

[2]We experimented with other threshold levels but we found 0.5 to provide an optimal balance between eliminating false positives and not ignoring similar sentences.

be matched with any sentence in the first version. If the unmatched sentence is surrounded by sentences that could be matched, then the editor only added a single sentence.

The fourth category is `missing_span`. This category is an extension of the previous category. Like in the previous category, the link was added by writing a new sentence and hyperlinking part of the text. However, in this case, the editors wrote a span of multiple sentences. As such, if the unmatched sentence is surrounded by unmatched sentences, then we consider the link to be added in a text span, not just a sentence.

The last category is `missing_section`. In this case, the links were added in a section that did not exist in the first version, which means the editor created a section, wrote the text in the section, and added the link. These links were detected by finding sentences with links in the second version present in a section that did not exist in the first version.

Table 3.2 shows an example for each of the link insertion categories, except for the category `missing_section`. As can be seen in the table, the problem of entity insertion grows in complexity as more text is missing. Additionally, Figure 1 shows the distribution of link insertion strategies for a subset of 21 languages. The majority of the added links fall in the `missing` categories, which further highlights the difficulty of the entity insertion task.

3.2 Two-stage dataset

The links extracted from the raw data can be used to train an entity insertion model, as each link is a representation of a unique entity related to the surrounding context. We built a two-stage training dataset from our collected links (see Section 4.3.3 for an in-depth discussion). The data for the first stage is built using the existing links, as described in Section 3.1.1, taken from the timestamp of 2023-10-01. The data for the second stage is built using the links added between 2023-09-01 and 2023-10-01. We additionally built a testing dataset using the links added between 2023-10-01 and 2023-11-01. Building such a test dataset has two advantages. First, it allows us to evaluate the entity insertion module's performance on all the link insertion categories. Secondly, defining the test data in this way makes it impossible for train data to leak into the test data, as the links used for testing did not yet exist during training.

For the existing links, we store the following data: source and target titles, QIDs and lead paragraphs, the name of the section containing the link, and a context surrounding the link. The context is defined as the sentence containing the link and the five sentences before and after (or until we reach the end of the section). We additionally keep positional information about the mention and the sentence containing the mention relative to the context (i.e., the start and end indices of the mention and the sentence in the context). The positional information is relevant to the data augmentation strategy we introduced (see Section 4.3.1). For the added links, we store the same information as in the first stage, except for the positional information, as the data augmentation strategy we introduced is only relevant to the first stage.

Table 3.2: **Examples of different link insertion strategies.** The added link is marked in blue.

Strategy	First Version Text	Second Version Text
Text Present	They also have white on the front part of the body. It has a distinctive prehensile tail that is coiled up and is used to help support the monkey on a branch.	They also have white on the front part of the body. It has a prehensile coiled tail. It is used to help support the monkey on a branch.
Missing Mention	Past famous winners include ABBA (1974), Céline Dion (1988), Lordi (2006), Loreen (2012 and 2023), Duncan Laurence (2019) and Måneskin (2021).	Past famous winners include ABBA (1974), Céline Dion (1988), Sandie Shaw (1967), Lordi (2006), Alexander Rybak (2009), Loreen (2012 and 2023), Conchita Wurst (2014), Netta Barzilai (2018), Duncan Laurence (2019) and Måneskin (2021).
Missing Sentence	November 23 – Lucas Grabeel, American actor, singer and songwriter December 7 – Robert Kubica, Polish Formula 1 driver	November 23 – Lucas Grabeel, American actor, singer, and songwriter November 28 – Mary Elizabeth Winstead, American actress December 7 – Robert Kubica, Polish Formula 1 driver
Missing Span	Mark Henry (born June 12, 1971) is an American professional wrestler. He is currently signed to World Wrestling Entertainment on its Legends contract. Henry is also a 1992 Olympian and winner of the Arnold Classic strongman competition.	Mark Henry (born June 12, 1971) is an American professional wrestler, powerlifter, weightlifter, and strongman. He is currently signed to All Elite Wrestling (AEW) where he works as a commentator, coach, and talent scout. Henry is also known for his time in the WWE, where he worked for 25 years. Henry also competed at the Olympics in 1992 and 1996.

Table 3.3: **Comparison between the number of links for stage 1 and stage 2.** Due to the size of the data to process for English Wikipedia, only a sample of links was taken for English. All other languages have the full amount of added links in stage 2.

Language	#(Stage 1 Links)	#(Stage 2 Links)	#(Stage 2 Links) / #(Stage 1 Links)
en	166.7M	192.0K*	0.0012
fr	85.1M	129.7K	0.0015
ja	60.6M	149.8K	0.0025
it	51.2M	101.2K	0.0020
pt	21.9M	53.7K	0.0024
cs	14.4M	26.9K	0.0019
sk	3.4M	7.2K	0.0021
ms	2.9M	6.3K	0.0022
cy	2.7M	1.3K	0.0005
simple	2.6M	9.4K	0.0036
kk	1.6M	2.7K	0.0017
ur	1.5M	14.5K	0.0097
uz	1.4M	12.4K	0.0088
af	1.4M	3.3K	0.0024
hi	1.2M	3.2K	0.0027
sw	1.0M	1.1K	0.0011
is	725.8K	1.6K	0.0022
gu	411.8K	197	0.0005
ga	387.8K	849	0.0022
kn	253.5K	1.1K	0.0043
ps	49.7K	53	0.0011

Table 3.3 shows the amount of data available for the first stage and the amount of data available for the second stage. The table shows that there is from two to three orders of magnitude more data in the first stage than there is in the second stage. This fact justifies the need for the first stage, even though the second stage more closely represents the true data for the entity insertion task, as ignoring all the existing links in favour of only using the added links would waste huge amounts of data, which is particularly impactful for smaller languages. Section 5.4.4 shows an analysis of the impact of both stages in downstream model performance.

4 Modelling Framework

In this chapter, we'll present our modelling framework for entity insertion. Our framework uses language models to capture linguistic and structural interactions between the candidate text span and the target page. We'll formally describe our modelling framework, and describe how we trained the models composing our entity insertion module. Finally, we'll present all the novelties we introduced into the base modelling framework and the rationale behind each modification.

4.1 Model architecture

Our proposed model architecture is described in Figure 4.1. We represent the target entity E_{tgt} via three textual features, the title T_{tgt}, the lead text L_{tgt} which is a short paragraph present in most Wikipedia articles, and optionally the list of mentions M_{tgt} previously associated with the target entity. For example, the list of mentions for the entity *List of Roman gods and goddesses* (Q1209773) in Simple English Wikipedia is "Ancient Roman deities", "List of Roman gods and goddesses", and "ancient Roman goddess".

The document in which to insert the entity is represented as a collection of potentially overlapping text spans $\mathcal{X}_{src} = [x_1, ..., x_D]$, where D is the number of text spans in the document X_{src} associated to the source entity E_{src}. In the following description, we'll focus on a single candidate, x, as the modelling framework processes each candidate text span independently. The candidate is represented via two textual features, the text t contained in the text span and optionally the title of the section s containing the text span in the source article.

Both the source section title s and the list of previously used mentions M_{tgt} are modifications of the base modelling framework, introduced via knowledge injection as described in Section 4.3.2. In its base form, the textual features described above are concatenated together into a single textual input as

$$\texttt{[CLS]}\ T_{tgt}\ \texttt{[SEP]}\ L_{tgt}\ \texttt{[SEP]}\ t\ \texttt{[SEP]}$$

The textual input is then tokenized into a sequence of tokens to produce the encoder input $\phi = \mathscr{T}(\texttt{[CLS]}\, T_{tgt}\, \texttt{[SEP]}\, L_{tgt}\, \texttt{[SEP]}\, t\, \texttt{[SEP]}) \in \mathbb{R}^T$, where $\mathscr{T}(\cdot)$ is the tokenizer operator and T is the number of tokens. We use a transformer-based encoder $\Gamma : \mathbb{R}^T \to \mathbb{R}^{T \times d}$ to convert the sequence of tokens ϕ into a sequence of contextual embeddings with the same length T, where d is the embedding dimension. We additionally follow the approach introduced by BERT (Devlin et al., 2019) and use the embedding of the `[CLS]` token as the global input representation. The global embedding of the tokenized textual input is a relevance embedding, which measures how related the candidate x is to the entity E_{tgt}. An MLP $\Lambda : \mathbb{R}^d \to \mathbb{R}$ produces a scalar relevance score from the relevance embedding produced by the encoder Γ.

The previous description can be formalised using the following equations. Let Γ be a transformer-based encoder and Λ an MLP. Additionally, $\mathscr{T}$ be a tokenizer associated with Γ and let $\theta = \theta_\Gamma \cup \theta_\Lambda$ be the union of the learnable parameter space of Γ, θ_Γ, and the learnable parameter space of Λ, θ_Λ. Then, a relevance score can be obtained from the textual input via

$$\phi = \mathscr{T}(\texttt{[CLS]}\, T_{tgt}\, \texttt{[SEP]}\, L_{tgt}\, \texttt{[SEP]}\, t\, \texttt{[SEP]}) \tag{4.1}$$

$$R = \Gamma\left(\phi; \theta_\Gamma\right) \tag{4.2}$$

$$r = \Lambda\left(R_{\texttt{[CLS]}}; \theta_\Lambda\right) \tag{4.3}$$

where $R \in \mathbb{R}^{T \times d}$ is the embedding for each token produced by the encoder, $R_{\texttt{[CLS]}} \in \mathbb{R}^d$ is the relevance embedding extracted from the `[CLS]`, and r is the relevance scalar used to rank the candidates.

At inference time, the procedure described above is applied to all the D candidate text spans, and a relevance score is obtained for each candidate. The candidate with the highest score is the candidate where the entity should be inserted.

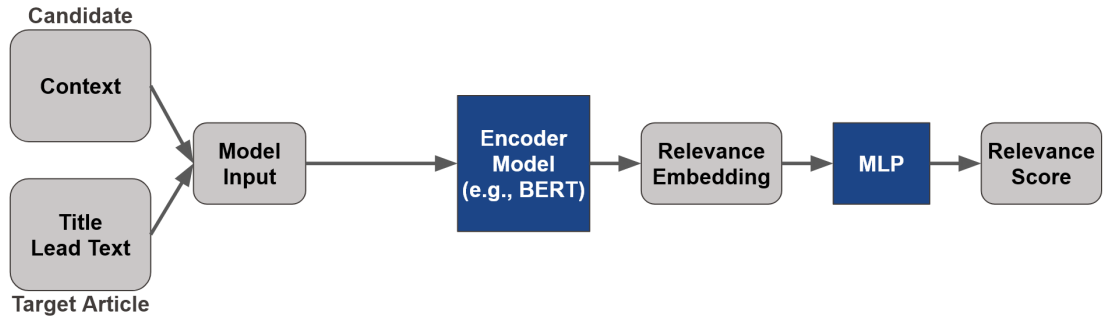
4.2 Training

We train the relevance scoring module using cross-entropy loss over a list of candidates. Given a target entity E_{tgt} and a list of candidate text spans $\mathscr{X}_N = [x_1, ..., x_N]$, where is N fixed for all lists as described in Section 5.1.1, there is only one true candidate in which to insert the entity. Let i' be the index of the correct candidate. Then, we employ the following list-wise objective to optimise the model.

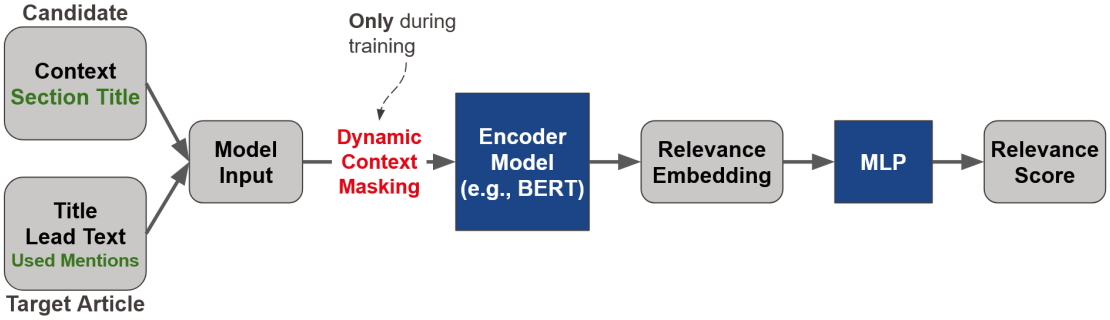
$$\max_\theta \frac{\exp\left(\text{score}\left(x_{i'}, E_{tgt}; \theta\right)\right)}{\sum_{i=1}^{N} \exp\left(\text{score}\left(x_i, E_{tgt}; \theta\right)\right)} \tag{4.4}$$

where score is an operator chaining the operations from Equations 4.1, 4.2, 4.3.

This objective function introduces the notion of ranking into the model because the model is not tasked with the binary task of "Is the text span relevant?" but rather with the ranking task of "Which text span is most relevant?". The ranking loss is more consistent with the ultimate end goal of an entity insertion model, as the model will be tasked with finding the best K text



(a) **Base modelling framework.**



(b) **Improved modelling framework.** Modifications in red are related to the novel data augmentation strategy, and modifications in green are related to the knowledge injection approach.

Figure 4.1: **Difference between the base modelling framework and the improved modelling framework.**

spans, which is implicitly a ranking task.

4.3 Introduced novelties

We introduce three modifications to the base modelling framework to improve model performance. We introduce a novel data augmentation strategy, titled dynamic context masking, we apply knowledge injection techniques to provide additional knowledge to the model, and we develop a two-stage training pipeline, to take full advantage of the two types of training links we have available, the existing links and the added links.

Figure 4.1a shows the standard modelling framework, and Figure 4.1b highlights the modifications introduced with the novel data augmentation strategy and the knowledge injection.

4.3.1 Data augmentation

As discussed in Chapter 3, a key limitation of using the existing links to train an entity insertion model is that the existing links only reflect added links in the `text_present` category, as the

mention to be linked is necessarily in the page containing the link. Using only the added links from previous months prevents this issue, but as discussed in Section 3.2, there are two to three orders of magnitude more existing links that there are added links, and removing the existing links would lead to a very large amount of wasted data.

We introduce a novel data augmentation strategy, titled dynamic context masking, which modifies the context associated with each existing link during training to simulate added links under different insertion strategies, namely `missing_mention`, `missing_sentence`, and `missing_span`. Dynamic context masking removes parts of the context surrounding a link, to simulate the editors' behaviour, where they simultaneously add more text to the article and add the link to part of the added text.

To simulate links in the `missing_mention` category, we remove only the link's mention from the context, which we can do because we stored the start and end indices of the mention relative to the context. To simulate links in the `missing_sentence` category, we remove the entire sentence containing the link, which we again can do because we stored the start and end indices of the link's sentence relative to the context. Finally, to simulate links in the `missing_span` category, we randomly select from one to three sentences before and after the link, and we remove those sentences alongside the sentence containing the link. Table 4.1 shows an example for each of the masking strategies.

During training, before sending the input to the model, we randomly select one of the masking strategies described above (as well as no masking) and we modify the input accordingly. However, before applying the strategy, we verify if the selected strategy does not produce an empty input. This may happen when, for example, the context is a single sentence, in which case simulating the `mask_sentence` insertion strategy would lead to an empty input. If the sampled strategy would produce an empty input, we re-sample a less aggressive strategy (in the previous example, we would select either the masking strategy for the `missing_mention` case or apply no masking, producing the default `text_present` case).

4.3.2 Knowledge injection

We use the lead paragraph and the title of the target article as the target's representation, and we use a span of sentences from the source article as the candidate in which we would like to insert the link. By providing these two sources of information to the model jointly, the model can capture linguistic and structural patterns between the two, and measure the target entity's relevance to the candidate text span. However, other sources of knowledge can be exploited to produce better relevance embeddings. In our work, we studied two of these.

First, we added the candidate's section title to the input. We believe that this knowledge better encapsulates the overall content of the candidate, and can help the model produce better representations of the input. For example, consider the context below.

> Given an angle ϕ, draw a ray from $(a,0)$ whose angle with the x-axis is ϕ. Draw a ray from the origin to the point where the first ray intersects the curve. Then, by the construction of the curve, the angle between the second ray and the x-axis is $\phi/3$.

Given this context, the model might struggle to capture the semantic meaning of the sentences. Instead, the section's title, "The trisection property", provides a summarised description of the context that may guide the model into producing better representations.

Then, we add the list of previously used mentions linked to the target entity. The motivation behind this knowledge injection is that the list of used mentions provides a strong signal of how the entity is typically referenced in the text and will allow the model to attend to these mentions when computing the relevance embedding.

To summarise, we inject multiple sources of knowledge into the input to produce better relevance embeddings. After the knowledge injections, the input takes the final form

$$\texttt{[CLS]}\, T_{tgt} M_{tgt}\, \texttt{[SEP]}\, L_{tgt}\, \texttt{[SEP]}\, s\, \texttt{[SEP]}\, t\, \texttt{[SEP]}$$

where we use the same notation as in Section 4.1 to describe each textual feature.

4.3.3 Two-stage training pipeline

We extract two types of links from the data: existing links and added links. As explained in Section 4.3.1, we introduce dynamic context masking to the existing links to simulate the conditions of `missing_mention`, `missing_sentence`, `missing_span`. However, this artificial data augmentation may generate structural and linguistic patterns in the data that would not occur in text written by human editors. For example, given the sentence "Laika was a Soviet space dog who was one of the first animals in space and the first to orbit the Earth", where the mention associated with the entity *Soviet space dogs* is underlined, applying the mention masking strategy would produce the sentence "Laika was a who was one of the first animals in space and the first to orbit the Earth". The masked sentence is unlikely to be found in natural text articles and as such there is a distribution shift from the augmented train data to the test data.

To reduce the impact of this shift, we introduced a two-stage training pipeline. Let m_0 be the month used for the Wikipedia timestamp to extract the links and m_{-1} be the previous month. In stage one, we use all the links extracted from the Wikipedia versions at m_0, modified with the dynamic context masking augmentation strategy. Then, in the second stage, we use all the links added between m_{-1} and m_0. For the second stage, we no longer need to use dynamic context masking, as we have access to real contexts used by editors to insert links.

Figure 4.2 shows the training loss evolution during a two-stage training run. The figure shows there is a clear distribution shift when moving from stage one to stage two (at step 10'000).

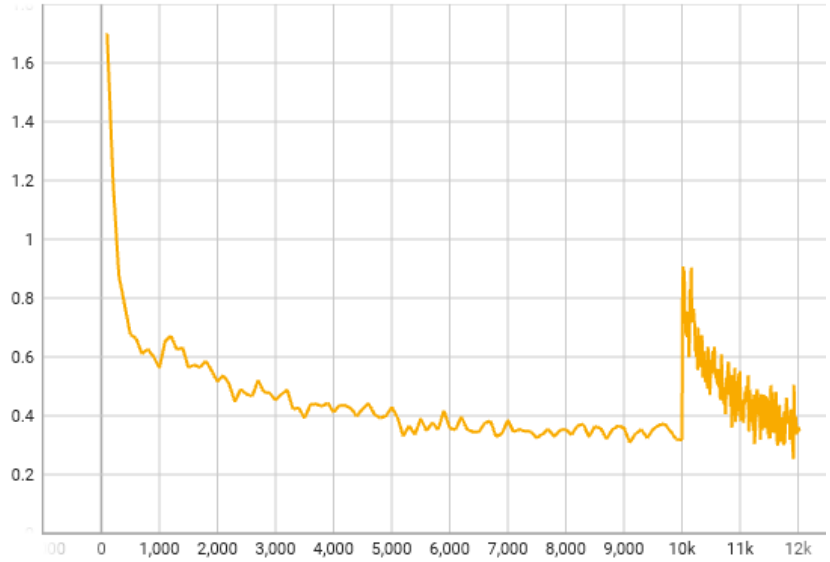


Figure 4.2: **Training loss for a two-stage training run.** Second stage started at step 10'000.

The spike in the loss indicates the model had not yet converged for the real data. Nevertheless, the performance does not return to the initial state and the loss converges quickly, which indicates that some knowledge transfer was possible from the first stage to the second.

Table 4.1: **Examples of different strategies for dynamic context masking.** The mention of the target link is marked in blue.

Strategy	Original Text	Modified Text
No Masking	Pulaski County is a county located in the central portion of the U.S. state of Georgia. As of the 2020 census, the population was 9,855. The county seat is Hawkinsville.	Pulaski County is a county located in the central portion of the U.S. state of Georgia. As of the 2020 census, the population was 9,855. The county seat is Hawkinsville.
Mention Masking	Perthes-lès-Brienne is a commune of the Aube département in the north-central part of France.	Perthes-lès-Brienne is a commune of the Aube in the north-central part of France.
Sentence Masking	In this Japanese name, the family name is Fujita. Yoshiaki Fujita (born 12 January 1983) is a Japanese football player. He plays for Oita Trinita.	In this Japanese name, the family name is Fujita. He plays for Oita Trinita.
Span Masking	Administration The department of French Guiana is managed by the Collectivité territorial de la Guyane in Cayenne. There are 2 arrondissements (districts) and 22 communes (municipalities) in French Guiana. The cantons of the department were eliminated on 31 December 2015 by the Law 2011-884 of 27 July 2011. The 22 communes in the department are:	Administration The 22 communes in the department are:

5 Experiments

In this chapter, we'll present the results obtained from our work. We'll start by presenting the data used to train and benchmark the model, as well as the baselines against which we'll compare our proposed model, and then present the full results for multilingual entity insertion. Afterwards, we'll discuss the results from an extensive list of ablations, justifying our modelling decisions.

We'd like to preface that our modelling framework is incompatible with the `missing_section` link insertion scenario, as we need the previous text from the same section and the section did not exist at editing time. The surrounding sections are not a good indicator for the insertion of a new entity, because they can cover vastly different subjects. As such, it is not clear how our modelling framework could be used in the `missing_section` scenario. One way of approaching this scenario would be to use a generative model to produce a new section when none of the existing candidates leads to a high relevance score. However, this falls outside the scope of our research and we leave this avenue to be explored in future work.

5.1 Data

As described in Section 3.2, we use a two-stage dataset to train the model. Each data point consists of a list of $N+1$ candidates, where one candidate is the true context containing the link, and the remaining N candidates are negative candidates.

The first stage is composed of existing links. When the dynamic context masking strategy is used, we apply the following distribution: 40% of the links are left unchanged, 20% of the links we mask the mention, 30% of the links we mask the sentence, and 10% of the links we mask a span of multiple sentences.

The second stage is composed of added links. Since these links naturally display the different link insertion strategies, no masking is applied to them.

5.1.1 Negative candidates

For the training links, for a given positive candidate, we extract N negative candidates. Whenever possible, we select N negative candidates from the same source article as the positive candidate. However, some pages are too small to be able to select N additional candidates. In those cases, we sample the remaining negative candidates randomly from other articles. As such, the number of "easy" negatives (taken from other articles) and "hard" negatives (taken from the same articles) will vary by language, because some languages have shorter pages.

For the testing links, we want to make the testing pipeline mimics as closely as possible to real usage of an entity insertion module for link insertion in Wikipedia. For that reason, we do not set a limit on the number of negative candidates, using all the available candidates on the page (other than the positive candidate).

We employ the following rules when constructing the negative candidates, both for training and testing: 1) a candidate's context will not span over two different sections, and 2) a candidate's context will not contain any of the mentions previously used to link to the target. The first rule keeps the content of each context consistent, as two distinct sections can cover very different topics. The second rule ensures that all the candidates used to evaluate the module are correctly classified as either positive candidates or negative candidates. For example, if the goal is to insert the entity "1984" (the book - Q208460) and there is a sentence in the article with the word "1984" not linked to the target article, there could be three reasons for the link to be missing. First, the mention "1984" could be related to a different entity (e.g., the year - Q2432), in which case the sentence should belong to a negative candidate. Second, the mention is supposed to be for the target entity but it is not yet linked and it should be, in which case the sentence should belong to an additional positive candidate. Third, the mention is supposed to be for the target entity, but it should not be linked because of Wikipedia's editing guidelines, in which case it is not clear whether the sentence should belong to a negative or a positive candidate. Because of this unclear categorisation, we choose to remove any sentences containing mentions associated with the target entity to insert.

5.2 Baselines

5.2.1 String match

A very simple baseline uses the list of previously used mentions and counts occurrences of the mentions in the candidate. We used the following implementation.

We employ a tokenizer $\mathcal{T}(\cdot)$ to extract keywords from the candidate and the list of mentions. The tokenizer is responsible for splitting the text into words and removing stopwords. Then, once we have a list of keywords for the candidate and a list of keywords for the mentions we compute the intersection of the keywords from the two sources, which produces a relevance score. Finally, we normalise the relevance score to avoid penalising shorter candidates. This

procedure is applied to all candidates (documents) for a given list of mentions (query) to produce a ranking of the candidates. As such, the string match baseline behaves as a surface form matcher.

Algorithm 1 formalises the string match implementation used in our work.

Algorithm 1 String match implementation.

Input: Candidate text c, List of mentions M, tokenizer $\mathscr{T}(\cdot)$, frequency counter $\mathscr{C}(\cdot,\cdot)$ (number of times the first argument occurs in the second)

Output: Relevance score r

$c_t = \mathscr{T}(c)$ ▷ Tokenize candidate text, length l_c
$M_t = \mathscr{T}(M)$ ▷ Tokenize list of mentions, length l_M
$r = 0$
for m in M_t **do**
    $r = r + \mathscr{C}(m, c_t)$ ▷ Count occurrences
end for
$r = r / l_c$ ▷ Divide by context size

5.2.2 BM25

BM25 is a ranking function that produces a relevance score for a set of documents given a search query. Given a document D and a query Q with keywords $q_1, ..., q_n$, one of the most popular BM25 variations computes a relevance score via

$$\text{score}(D,Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i,D)\cdot(k_1+1)}{f(q_i,D) + k_1 \cdot \left(1 - b + b\frac{|D|}{avgdl}\right)} \tag{5.1}$$

where $f(q_i,D)$ is the number of times q_i occurs in D, $|D|$ is the length of D in words, and $avgdl$ is the average document length in the corpus from which documents are drawn. k_1 and b are free parameters, usually chosen as $k_1 \in [1.2, 2.0]$ and $b = 0.75$ (Manning et al., 2012).

In our work we used the Okapi-BM25 implementation as presented in Trotman et al. (2014). In this variant, the IDF term is defined as

$$\text{IDF}(q_i) = \log\left(\frac{N - D_{q_i} + 0.5}{D_{q_i} + 0.5}\right) \tag{5.2}$$

where N is the size of the corpus from which documents are drawn and D_{q_i} is the number of documents containing q_i.

Unlike in the string match baseline, for BM25 we extract keywords from the list of mentions and the target lead. We believe that this provides more information to BM25, and distinguishes it from the string match baseline.

5.3 Multilingual entity insertion

We evaluated our model for entity insertion against 20 of the considered languages. Pashto (ps) was used for training the multilingual model but, since there were only 10 added links, we excluded it from the evaluation as the results would not be statistically significant.

We compared our model against the string match and BM25 baselines described above, and against a random ranker. Additionally, we compared the performance of three model variants:

- m^0_{LANG}: Family of monolingual models (one model per language) fine-tuned with none of the injections/augmentations and only with the first stage;
- m_{LANG}: Family of monolingual models (one model per language) fine-tuned with all the novelties introduced in Section 4.3;
- M_{LARGE}: Multilingual model fine-tuned on all the languages with all the novelties introduced in Section 4.3.

Figure 5.1 shows the performance of the entity insertion modules described for each language, and Table 5.1 shows the aggregated scores for all methods.

One key finding is that the string match and BM25 baselines perform much worse for any of the `missing` cases than for the `text_present` case. BM25 performs better than string match, which shows that the information in the target entity's lead paragraph is useful. Still, we even see that in several languages the performance is close to random for the `missing` cases. These findings justify the need for more complex entity insertion tools, as the classical baselines are insufficient for the harder cases.

Comparing m^0_{LANG}, m_{LANG}, and M_{LARGE}, we see that naively fine-tuning a language model for the entity insertion task is not sufficient, as m^0_{LANG} behaves only slightly better than BM25 for most languages. However, the most important finding is that the multilingual model M_{LARGE} consistently does better than or as well as m_{LANG}. This means that the multilingual XLM-RoBERTa is capable of transferring knowledge between the multiple languages it was trained on to improve overall performance. This gap is particularly big for smaller resource languages, like Welsh (cy), Uzbek (uz) or Afrikaans (af), as was to be expected, but the trend is also visible in high resource languages, such as English (en) and French (fr). Even for these large languages, the model can exploit multilingual knowledge and transfer knowledge from other languages to improve performance. Furthermore, we see that from Table 5.1 that the drop from the `text_present` cases to the `missing` cases is much less sharp for the multilingual model than for both the monolingual family of models and the classical baselines.

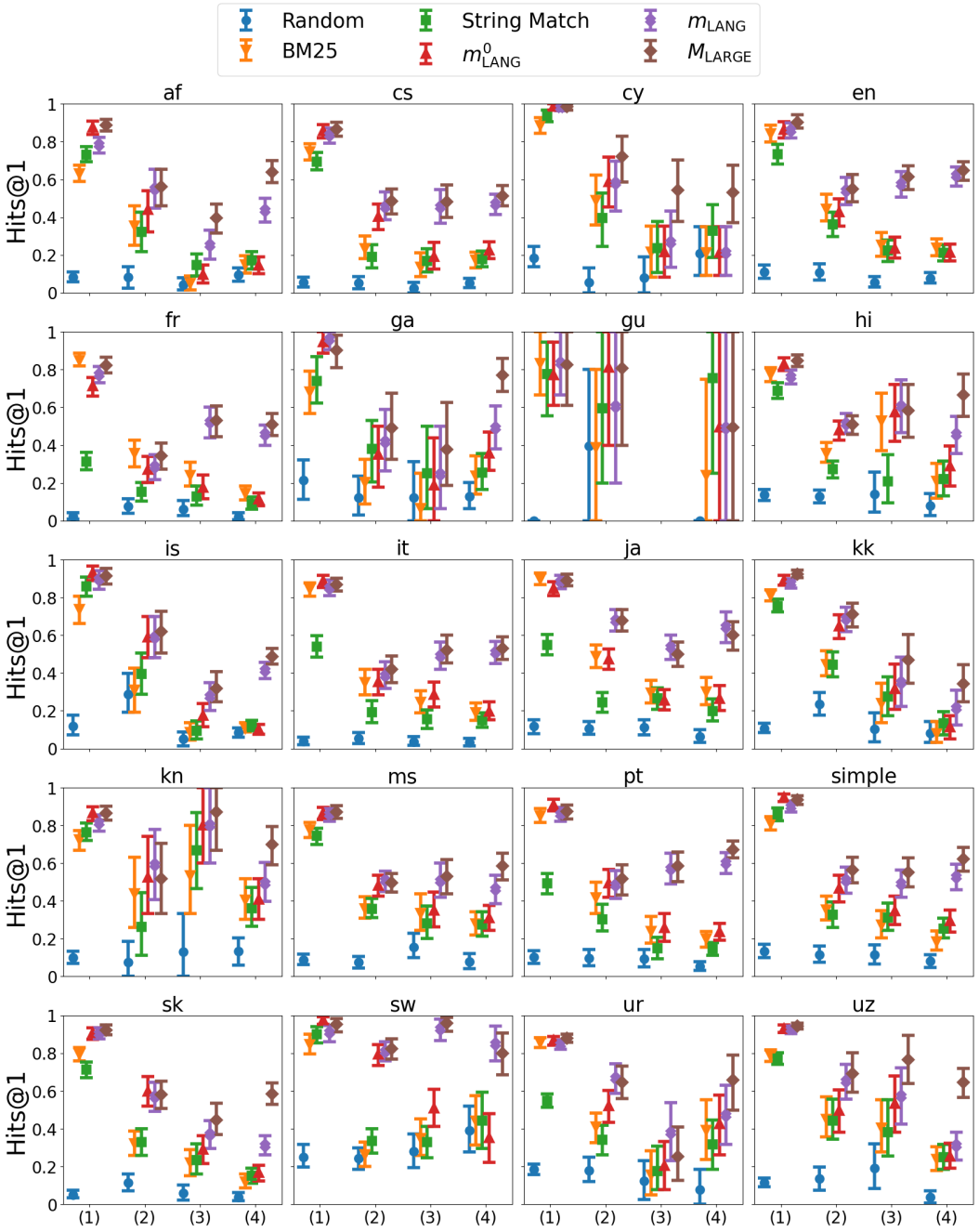


Figure 5.1: **Multilingual results.** m_{SIMPLE} is a family of monolingual models fine-tuned with only the first stage and no novelties, m_{FULL} is a family of monolingual models fine-tuned with both stages and the introduced novelties, and M_{FULL} is a multilingual model fine-tuned with both stages and the introduced novelties. Error bars represent a 95%CI. x-axis legend is (1): `text_present`, (2): `missing_mention`, (3): `missing_sentence`, (4): `missing_span`.

5.3.1 Zero-shot vs fine-tuned

In this section, we'll study the zero-shot performance of the multilingual model, i.e., the performance of the model on languages it was not fine-tuned on. To study the zero-shot

Table 5.1: **Aggregate multilingual results.** Results obtained by macro-averaging over the 20 languages in Figure 5.1. Best scores with † are statistically different (p-value < 0.05) from the second-best score.

Method	Hits@1			MRR		
	Overall	Present	Missing	Overall	Present	Missing
Random	0.107	0.110	0.111	0.240	0.253	0.236
String Match	0.463	0.705	0.277	0.559	0.772	0.399
BM25	0.511	0.800	0.283	0.613	0.867	0.422
m^0_{LANG}	0.586	0.883	0.363	0.653	0.907	0.466
m_{LANG}	0.670	0.866	0.508	0.742	0.897	0.616
M_{LARGE}	**0.725**	**0.895**	**0.591**†	**0.791**	**0.922**	**0.693**†

Table 5.2: **Relative average performance of the models in Figure 5.2.** A score <100% means the first model was worse than the second and a score >100% means the first model was better than the second. Results obtained by macro-averaging over the 9 languages in Figure 5.2.

Comparison	Text Present	Missing Mention	Missing Sentence	Missing Span
M_{SMALL} vs. M_{LARGE}	98.8%	96.4%	98.6%	87.2%
M_{SMALL} vs m_{LANG}	102.4%	106.5%	116.6%	129.7%

performance of the model, we consider the model M_{LARGE} as introduced in the previous section, which is a multilingual model trained on all 21 languages in Table 3.1. We also introduce M_{SMALL}, which is a multilingual model trained on a subset of 12 languages, as described in Table 5.3. Finally, we considered the family of monolingual models m_{LANG} also introduced in the previous section. As discussed in Section 5.4.3, we used a small amount of data for the first stage, and all the available added links for the second stage, which explains why M_{LARGE} was trained on a larger amount of data than M_{SMALL} for the second stage.

We measured the performance of each of the models M_{LARGE}, M_{SMALL} and all the monolingual models on the languages on which M_{SMALL} was not trained on. Figure 5.2 shows the obtained results. The figure shows that M_{LARGE} and M_{SMALL} perform similarly. Considering Table 5.2 we see that the zero-shot model retains 95%+ model performance in all cases except the harder `missing_span`, where 87% model performance is retained. Additionally,we see the family of monolingual models m_{LANG} performs generally worse than either M_{LARGE} and M_{SMALL}.

The results from Figure 5.2 are a key finding to prove the usability of our entity insertion model. They show that the model is capable of transferring the knowledge acquired during fine-tuning to languages unseen during fine-tuning while maintaining a similar level of performance. Given the large number of languages in Wikipedia, showing knowledge transfer capabilities is vital because it is unrealistic to assume the model can be fine-tuned on all the available languages. We have additional confirmation that multilingual knowledge during training improves performance when compared with monolingual training, as even a zero-shot multilingual model outperforms a monolingual model.

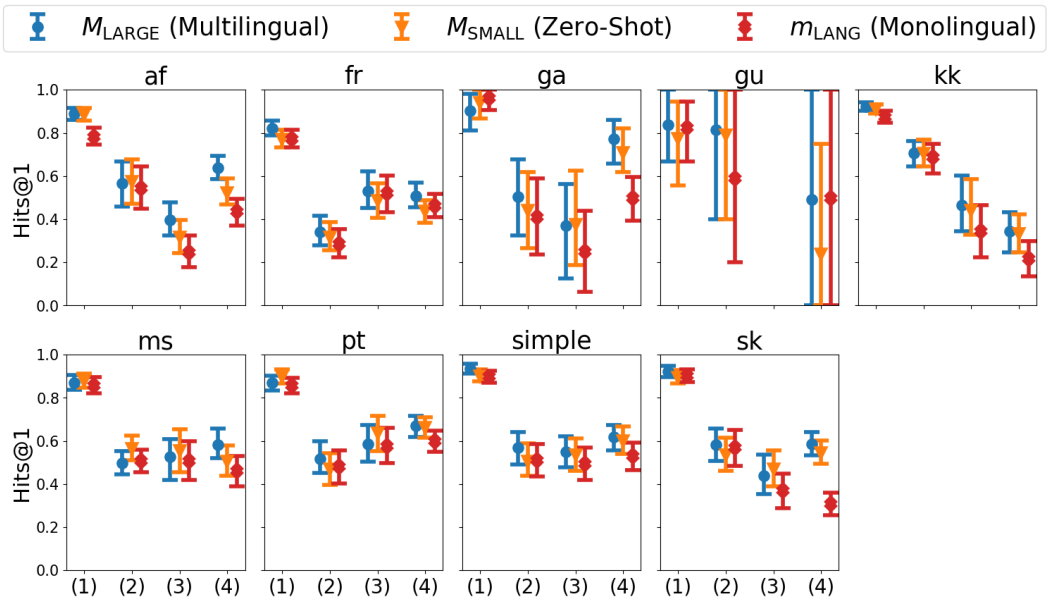


Figure 5.2: **Comparison of zero-shot performance with monolingual and multilingual performance.** M_{LARGE} was trained on all available languages, M_{SMALL} was trained on all languages except the inference languages, and m_{LANG} is a family of monolingual models, each model trained on its respective inference language. Error bars represent a 95%CI. x-axis legend is (1): `text_present`, (2): `missing_mention`, (3): `missing_sentence`, (4): `missing_span`.

5.4 Ablations

In this section, we perform an ablation of various hyperparameters and training configurations. We study the impact of the starting pre-trained model as well as the model size, the amount of training data needed for each stage, and the impact of the modifications we introduced in downstream performance.

Table 5.4 describes key statistics for all the different models considered in the ablations.

5.4.1 Starting model comparison

Since our approach is based on fine-tuning pre-trained models, the starting pre-trained model can impact model performance. We decided to investigate this dependence using three pre-trained models: $\text{BERT}_{\text{BASE}}$, $\text{RoBERTa}_{\text{BASE}}$ and the encoder portion of T5_{BASE}, which we call $\text{T5}^{\text{enc}}_{\text{BASE}}$. We considered BERT and RoBERTa because they are amongst the most popular transformer encoder models. We additionally included T5 to see how encoder-decoder models perform in the entity insertion task. However, as RankT5 (Zhuang et al., 2023) showed there was no clear benefit in using the full encoder-decoder architecture, as opposed to encoder-only architecture, for computational reasons we decided to use the encoder-only variant of T5, T5^{enc}.

Table 5.3: **Comparison between the two multilingual model variants, M_{LARGE} and M_{SMALL}.**

Model	Starting Model	Fine-Tuned Languages	Training Data Size	
			Stage 1	Stage 2
M_{LARGE}	XLM-RoBERTa$_{\text{BASE}}$	en, fr, it, ja, pt, cs, ms, cy, sk, uz, simple, kk, ur, hi, af, sw, ga, is, kn, gu, ps	20K	503K
M_{SMALL}	XLM-RoBERTa$_{\text{BASE}}$	en, it, ja, cs, cy, uz, ur, hi, sw, is, kn, ps	20K	348K

Table 5.4: **Relevant statistics for different models considered.** *HF Name* corresponds to the HuggingFace model name used to load the pre-trained model.

Model	HF Name	#Parameters	Tokenizer Size	#Layers	#Heads	Hidden Size
BERT$_{\text{BASE}}$	`base-base-cased`	108.3M	29.0K	12	12	768
T5$^{\text{enc}}_{\text{BASE}}$	`t5-base`*	109.6M	32.1K	12	12	768
RoBERTa$_{\text{BASE}}$	`roberta-base`	124.6M	50.3K	12	12	768
RoBERTa$_{\text{LARGE}}$	`roberta-large`	355.4M	50.3K	24	16	1024
XLM-RoBERTa$_{\text{BASE}}$	`xlm-roberta-base`	278.0M	250.0K	12	12	768

We trained each model on the Simple English dataset, and we measured their performance on the test data. Table 5.5 shows that the RoBERTa model outperformed both BERT and T5^{enc} in all link insertion categories by a large margin. BERT and T5^{enc} performed similarly, with T5^{enc} doing slightly better. Considering the model descriptions in Table 5.4, two likely explanations exist for the observed results. First, we see that the RoBERTa model has 15% more parameters than T5^{enc} and BERT. A larger model can capture more complex interactions between the candidate and the target, and that could explain the performance boost seen in RoBERTa. Second, the RoBERTa tokenizer has a much larger vocabulary than the tokenizers for BERT or T5^{enc}. A larger vocabulary might make it possible for the model to capture more fine-grained linguistic and structural patterns in the candidate text, enabling the model to exploit patterns that neither T5^{enc} nor BERT can capture. In the following section, we study the impact of model size on performance to determine which of the two hypotheses is most likely to be true.

We also see that the models behave differently regarding the different link insertion strategies. All models behave very strongly in the `text_present` case, which is the easiest case. However, we see that RoBERTa seems to struggle in the `missing_mention` case much more than in the `missing_sentence` or `missing_span` cases, performing similarly in the latter two and 15% worse in `missing_mention`. On the other hand, both BERT and T5^{enc} struggle in the `missing_sentence` case, while the performance on `missing_mention` and `missing_span` is similar.

5.4.2 Model size comparison

There is a widely known trend in the deep learning community that bigger models tend to perform better than smaller models (Soltanolkotabi et al., 2019; Brutzkus and Globerson, 2019;

Table 5.5: **Entity insertion performance for different starting models.** Results obtained for Simple English. Scores with † are statistically significant (p-value < 0.05).

Model	Text Present		Missing Mention		Missing Sentence		Missing Span	
	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR
$BERT_{BASE}$	.916	.939	.570	.659	.403	.518	.499	.611
$T5^{enc}_{BASE}$	.928	.951	.565	.656	.501	.595	.599	.690
$RoBERTa_{BASE}$	**.956**†	**.968**†	**.695**†	**.757**†	**.835**†	**.881**†	**.861**†	**.860**†

Table 5.6: **Entity insertion performance for different model sizes.** Results obtained for Simple English. None of the scores are statistically significant.

Model	Text Present		Missing Mention		Missing Sentence		Missing Span	
	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR
$RoBERTa_{BASE}$	.956	.968	**.696**	**.760**	.834	.884	.799	.859
$RoBERTa_{LARGE}$	**.964**	**.975**	.670	.744	**.856**	**.895**	**.822**	**.873**

Simon et al., 2023). As such, we studied how the model size impacts performance for the entity insertion task, by comparing $RoBERTa_{LARGE}$ with $RoBERTa_{BASE}$ on the Simple English dataset.

Table 5.6 shows that there is no statistically significant difference between the performance of $RoBERTa_{LARGE}$ and $RoBERTa_{BASE}$. These results point to the fact that increased model complexity will not be enough to improve model performance. It is worth noting that these results were obtained for Simple English. The multilingual problem is much harder and it might benefit from the increased complexity of the large model. We leave this study for future work.

Additionally, these findings give more strength to the hypothesis that the reason why RoBERTa is significantly better than BERT and $T5^{enc}$ is because RoBERTa's larger tokenizer allows the model to capture more fine-grained linguistic and structural patterns in the candidate. This increased input representation space seems to be vital for entity insertion.

5.4.3 Training size

We have a dataset of existing links for the first stage and a dataset of added links for the second stage. In this experiment, we studied how much data is needed for each stage. To study the model performance dependence on training data size, we trained a $RoBERTa_{BASE}$ model with different amounts of data of the full English dataset.

Figure 5.3a shows the performance of the model for the different link insertion categories for a stage 1 training dataset of sizes $\{10^3, 10^4, 10^5, 10^6\}$. For this plot, the model was trained using only the first stage. Figure 5.3b shows an analogous plot for stage 2, with training sizes of $\{10^2, 10^3, 10^4, 10^5\}$. For this plot, the model was trained using only the second stage.

These results show that it is much more important to have a long second stage than a long first



(a) **Impact of the amount of data for the first stage on model performance.** Model trained only using the first stage.



(b) **Impact of the amount of data for the second stage on model performance.** Model trained only using the second stage.

Figure 5.3: **Comparison of the impact of the amount of data on model performance for the two training stages.**

stage. The performance did not visibly improve over the data range considered for the first stage, indicating no benefit in training on a lot of existing links. On the other hand, the model performance improved drastically as the data size increased for the second stage, with no sign of plateauing. Based on these results, the optimal training schedule for an entity insertion model using our data seems to be a short first stage, followed by a second stage using as much data as possible.

5.4.4 Impact of training stages

Our full training pipeline includes two training stages. In the first stage, we use existing links, which we augment via dynamic context masking. In the second stage, we use added links, which do not need to be augmented because they reflect true link insertion strategies. To study the impact of each stage in performance, we conducted a study where we trained four

model variants.

1. Trained only with the first stage;
2. Trained only with the second stage;
3. Trained with both stages, without dynamic context masking on the first stage;
4. Trained with both stages, with dynamic context masking on the first stage.

We considered four different languages to train and evaluate the models. We used Kannada (253.5K links for the first phase, 1.1K links for the second phase), Uzbek (1.5M, 14.5K), Czech (14.4M, 26.9K), and English (166.7M, 192.0K). For all languages, we used 20K links for the first phase and all available links for the second phase. Finally, we trained a family of monolingual models, with one model for each language, and a single multilingual model for all languages. The multilingual model was also trained with 20K links for the first phase, and all available links for the second phase. Table 5.7 shows the results for the family of monolingual models, and Table 5.8 shows the results for the multilingual model.

Monolingual analysis

Table 5.7 shows that, for monolingual models, the first stage is vital for small languages, but for larger languages (where the second stage gets longer) the second stage becomes more important.

The results for Kannada, the smallest language, prove that when only a small amount of added links is available, it is not enough to train on them, and it is necessary to use the existing links, as applying only the first stage greatly outperforms applying only the second stage. However, using both stages is key to maximising overall performance. The first phase provides the necessary volume of links to train the model for the entity insertion task, and the second phase shifts the domain to the real link insertion distribution.

On the other hand, the results for English, the largest language, show that when a large amount of added links is available, the existing links are not necessary to fine-tune a language model for entity insertion. The performance in the `text_present` scenario is similar for all four variants, but the second stage greatly improves the performance for all the `missing` categories. The results also show that there is no clear winner between any of the variants using the second stage, leading to the conclusion that the model does not use the knowledge from the first stage when the second stage is large enough.

Multilingual analysis

For the multilingual model results in Table 5.8, we no longer see a difference in behaviour between the small languages and the large languages. Instead, applying the second stage

benefited all the languages. These results are consistent with the fact that the model is capable of using the knowledge acquired from all languages to increase the overall performance. As such, even though there was the same amount of added links for Kannada, training with all the added links from English, Czech, Uzbek, and Kannada improved the model's entity insertion ability beyond the explicit knowledge available in the Kannada added links.

In the multilingual case, we see the same trend as in the monolingual case for large languages. The second phase is much more important than the first phase, which is explained by the large amount of added links the multilingual model was trained on, albeit in multiple languages. There is again no clear winner between each of the variants including the second phase.

Table 5.7: **Impact of the different training stages for the monolingual models.** The stage variations are (1): only the first stage, (2): only the second stage, (3): first stage (without context masking) and second stage, (4): first stage (with context masking) and second stage. Scores with † are statistically greater (p-value < 0.05) than variant (1). Scores with * are statistically greater (p-value < 0.05) than variant (2).

		Text Present		Missing Mention		Missing Sentence		Missing Span		Average Missing		Overal Average	
		Hits@1	MRR	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR
kn	(1)	**0.863***	**0.882***	0.515	0.619	0.800*	0.875*	0.408	0.558	0.479*	0.607*	0.752*	0.805*
	(2)	0.283	0.401	0.367	0.498	0.331	0.459	0.287	0.516	0.311	0.504	0.290	0.433
	(3)	0.853*	0.876*	**0.627***	**0.712***	0.869*	0.902*	**0.579**$^{\dagger *}$	**0.736**$^{\dagger *}$	**0.622**$^{\dagger *}$	**0.749**$^{\dagger *}$	**0.786***	**0.839***
	(4)	0.810*	0.842*	0.597	0.674	**0.932***	**0.947***	0.489*	0.656$^{\dagger *}$	0.567*	0.695$^{\dagger *}$	0.740*	0.799*
uz	(1)	**0.929***	**0.945**	0.490	0.622	0.535	0.597	0.254	0.356	0.369	0.469	0.770	0.809
	(2)	0.901	0.928	0.618	0.703	0.553	0.654	0.368†	0.480†	0.468†	0.574†	0.778	0.827
	(3)	0.922	0.942	0.604	0.703	**0.646**	**0.735**	**0.404**†	**0.543**†	**0.503**†	**0.617**†	**0.802**†	**0.850**†
	(4)	0.924	0.942	**0.638**†	**0.729**†	0.620	0.699	0.321	0.452†	0.460†	0.572†	0.791	0.837†
cs	(1)	**0.857***	**0.873***	0.401	0.490	0.194	0.299	0.226	0.323	0.269	0.366	0.475	0.544
	(2)	0.788	0.830	**0.539**†	**0.630**†	0.414†	0.529†	0.405†	0.529†	0.447†	0.557†	0.567†	0.654†
	(3)	0.835	0.865	0.455	0.570†	0.413†	**0.532**†	0.433†	0.561†	0.435†	0.557†	0.575†	0.664†
	(4)	0.826	0.863	0.452	0.580†	**0.422**†	**0.532**†	**0.471**†	**0.577**†	**0.458**†	**0.569**†	**0.585**†	**0.672**†
en	(1)	0.864	0.887	0.429	0.526	0.234	0.337	0.213	0.314	0.276	0.377	0.436	0.515
	(2)	0.870	0.896	**0.549**†	0.620†	0.607†	0.676†	**0.638**†	**0.723**†	**0.604**†	**0.682**†	**0.678**†	**0.741**†
	(3)	**0.885**	**0.911**	0.503	0.605†	**0.629**†	**0.693**†	0.609†	0.710†	0.586†	0.677†	0.668†	0.740†
	(4)	0.863	0.893	0.544†	**0.630**†	0.592†	0.662†	0.612†	0.698†	0.588†	0.670†	0.664†	0.732†

5.4.5 Impact of introduced novelties

To study the impact of the introduced novelties, we measured the entity insertion performance of the model as we added each novelty one at a time. As such, we considered five model variants.

1. Simple fine-tuning, with only the first stage and none of the introduced novelties;
2. (1) + second stage;
3. (2) + dynamic context masking;
4. (3) + source section knowledge;
5. (4) + target mentions knowledge;

Table 5.8: **Impact of the different training stages for the multilingual model.** The stage variations are (1): only the first stage, (2): only second stage, (3): the first stage (without context masking) and the second stage, (4): the first stage (with context masking) and second stage. Scores with † are statistically greater (p-value < 0.05) than variant (1). There is no statistical difference between any of the (2), (3) and (4) scores.

		Text Present		Missing Mention		Missing Sentence		Missing Span		Average Missing		Overall Average	
		Hits@1	MRR	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR
kn	(1)	**0.874**	**0.892**	0.560	0.641	0.798	0.839	0.434	0.566	0.500	0.618	0.765	0.812
	(2)	0.854	0.879	**0.697**	**0.753**	0.931	0.946	0.647†	0.812†	0.695†	0.817†	0.808	0.861†
	(3)	0.868	0.890	0.631	0.731	0.930	**0.966**	**0.697**†	0.824†	0.711†	0.818†	**0.820**†	**0.869**†
	(4)	0.824	0.860	0.656	0.750	**0.935**	0.957	0.694†	**0.839**†	**0.720**†	**0.834**†	0.794	0.853†
uz	(1)	0.911	0.933	0.569	0.670	0.469	0.561	0.224	0.333	0.360	0.469	0.753	0.800
	(2)	0.942†	0.957†	**0.689**	**0.773**†	**0.812**†	**0.850**†	**0.629**†	**0.733**†	**0.677**†	**0.764**†	**0.867**†	**0.902**†
	(3)	0.942†	**0.958**†	0.686	0.769†	0.763†	0.823†	0.592†	0.708†	0.647†	0.744†	0.858†	0.897†
	(4)	**0.945**†	0.957†	0.676	0.748	0.787†	0.842†	0.578†	0.714†	0.641†	0.744†	0.859†	0.897†
cs	(1)	0.851	0.879	0.432	0.525	0.225	0.308	0.242	0.313	0.294	0.372	0.490	0.550
	(2)	0.859	0.884	**0.531**†	**0.624**†	**0.485**†	**0.591**†	**0.507**†	**0.621**†	**0.510**†	**0.615**†	**0.633**†	**0.708**†
	(3)	**0.871**	**0.896**	0.512	0.623†	0.423†	0.550†	0.472†	0.592†	0.473†	0.593†	0.613†	0.700†
	(4)	0.838	0.875	0.480	0.598†	0.461†	0.561†	0.474†	0.607†	0.475†	0.596†	0.601†	0.694†
en	(1)	0.859	0.887	0.450	0.537	0.238	0.340	0.222	0.331	0.290	0.389	0.443	0.524
	(2)	0.888	0.910	0.519	0.606	0.577†	0.665†	**0.674**†	**0.741**†	0.605†	0.684†	0.682†	0.746†
	(3)	0.896	0.925†	**0.539**†	0.620†	0.601†	0.677†	0.658†	**0.741**†	**0.609**†	**0.691**†	**0.687**†	0.755†
	(4)	**0.908**†	**0.928**†	0.538†	**0.628**†	**0.608**†	**0.687**†	0.641†	0.730†	0.604†	0.690†	0.686†	**0.756**†

We measured the performance of each of the variants in the same four languages as in Section 5.4.4, Kannada, Uzbek, Czech, and English. Table 5.9 shows the results obtained for each language with a multilingual model. We found the multilingual model presented the same general trends as the family of monolingual models, so we omitted the monolingual results and discuss only the multilingual results.

The key result from these experiments is that the addition of the second stage is the most important modification introduced to the training pipeline, improving performance by 0.2-0.3. This finding further demonstrates the semantic gap between the augmented data from the existing links and the true data from the added links.

The addition of the remaining novelties had a smaller impact on model performance. Nevertheless, we see that in general, they lead to an improvement in performance. Variant (5), which is the model using all the novelties, was the best-performing variant 56% of the time. In contrast, variant (2), which only exploits the two-stage training pipeline, was the best-performing variant 12.5% of the time.

The overall improvement from the introduced novelties is most significant from the smaller languages. This suggests that, even with knowledge transfer from multiple languages, the lack of language-specific data forces the model to take advantage of the additional information provided. For larger languages, the large amount of data seems to be sufficient and additional knowledge is not necessary.

It should be noted that, for all languages, all the variants containing novelties (i.e., all variants except variant (1)) produce scores which are not statistically different. For smaller languages,

Table 5.9: **Impact of the introduced novelties on performance.** The novelty variations are (1): simple fine-tuning (no novelties), (2): (1) + second stage, (3): (2) + dynamic context masking, (4): (3) + source section knowledge, (5): (4) + target mentions knowledge. Scores with † are statistically greater (p-value < 0.05) than the previous variant. Scores with * are statistically greater (p-value < 0.05) than variant (2).

		Text Present		Missing Mention		Missing Sentence		Missing Span		Average Missing		Overall Average	
		Hits@1	MRR	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR
kn	(1)	**0.874**	**0.890**	0.555	0.640	0.798	0.838	0.433	0.568	0.503	0.614	0.765	0.812
	(2)	0.862	0.884	0.626	0.724	0.870	0.917	0.674†	0.813†	0.686†	0.807†	0.809	0.861†
	(3)	0.836	0.863	0.514	0.647	0.797	0.839	0.710	0.828	0.681	0.788	0.791	0.843
	(4)	0.843	0.870	0.624	0.720	0.932	0.952	0.719	0.849	0.728	0.830	0.810	0.856
	(5)	0.834	0.864	**0.632**	**0.762**	**1.000**	**1.000**	**0.785**	**0.871**	**0.776**	**0.865***	**0.817**	**0.864**
uz	(1)	0.912	0.933	0.567	0.670	0.468	0.557	0.224	0.334	0.362	0.466	0.753	0.800
	(2)	0.921	**0.943**	0.607	0.705	0.784†	**0.833**†	0.577†	0.695†	0.622†	0.722†	0.835†	0.880†
	(3)	0.921	0.942	0.654	0.720	0.765	0.829	0.598	0.694	0.642	0.725	0.841	0.880
	(4)	0.921	0.939	0.594	0.691	0.745	0.820	0.631	0.726	0.637	0.731	0.841	0.880
	(5)	**0.922**	0.942	**0.655**	**0.742**	**0.790**	0.827	**0.640**	**0.744**	**0.670**	**0.754**	**0.850**	**0.890**
cs	(1)	0.851	0.879	0.435	0.526	0.224	0.306	0.244	0.310	0.294	0.372	0.490	0.550
	(2)	0.844	0.877	0.555†	0.642†	0.427†	0.546†	0.487†	0.601†	0.495†	0.602†	0.618†	0.700†
	(3)	**0.867**	**0.889**	**0.593**	**0.668**	0.444	0.549	0.478	0.587	**0.505**	0.604	**0.632**	**0.703**
	(4)	0.829	0.864	0.515	0.619	0.445	0.560	0.475	0.598	0.481	0.596	0.602	0.691
	(5)	0.834	0.869	0.482	0.595	**0.506**	**0.606**	**0.504**	**0.611**	0.500	**0.607**	0.617	0.699
en	(1)	0.860	0.886	0.450	0.538	0.237	0.341	0.223	0.332	0.288	0.390	0.443	0.524
	(2)	0.873	0.898	0.516†	0.614†	0.582†	0.673†	**0.645**†	**0.725**†	**0.594**†	**0.680**†	**0.669**†	**0.741**†
	(3)	**0.889**	**0.907**	**0.524**	**0.624**	0.583	0.666	0.621	0.705	0.584	0.671	0.667	0.736
	(4)	0.849	0.885	0.510	0.609	0.554	0.649	0.607	0.696	0.567	0.661	0.644	0.721
	(5)	0.878	0.903	0.507	0.601	**0.607**	**0.680**	0.630	0.711	0.590	0.674	0.668	0.736

the lack of statistical relevance can be explained by the lack of data. For example, for Kannada, only 14 test links fall in the category of `missing_sentence`. As such, the value statistics become less relevant, which explains why large improvements of 10-15% are not considered statistically significant. For larger languages, the lack of statistical relevance again points to the fact that the introduced modifications have a reduced impact when the model has access to a larger amount of data.

5.4.6 Multilingual model vs monolingual model

In our ablations, we found the scores obtained with RoBERTa on Simple English to be significantly higher than the scores achieved by the multilingual XLM-RoBERTa. In this section, we compare the performance of these two models on the full English dataset, with both models having been fine-tuned on the same English dataset. Table 5.10 shows there is a significant difference in the performance of RoBERTa and XLM-RoBERTa, with RoBERTa scoring higher in all link insertion strategies, with gaps up to 25%. We draw two conclusions from these results.

In our ablations, we found that RoBERTa outperformed BERT and T5^{enc} by a large margin, which leads us to select XLM-RoBERTa as the best candidate for the multilingual model to use in our experiments. However, the performance of RoBERTa does not seem to directly correlate with the performance of XLM-RoBERTa, as seen by the large drop in English when moving from RoBERTa to XLM-RoBERTa. This finding casts some doubt on the decision of the best

Table 5.10: **Entity insertion performance for a monolingual model and a multilingual model.** $\text{RoBERTa}_{\text{BASE}}$ was pre-trained in English. $\text{XLM-RoBERTa}_{\text{BASE}}$ was pre-trained on 100 languages. Both models were fine-tuned on our English dataset. Scores with † are statistically significant (p-value < 0.05).

Model	Text Present		Missing Mention		Missing Sentence		Missing Span	
	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR	Hits@1	MRR
$\text{RoBERTa}_{\text{BASE}}$	**.923**†	**.936**†	**.737**†	**.797**†	**.850**†	**.898**†	**.787**†	**.848**†
$\text{XLM-RoBERTa}_{\text{BASE}}$	.863	.892	.543	.630	.595	.662	.697	.615

multilingual model and opens the doors to models like multilingual BERT and mT5 (Xue et al., 2021). In future work, it would be interesting to consider other multilingual models and see if they can outperform XLM-RoBERTa.

More importantly, we remember from Section 5.3 that XLM-RoBERTa fine-tuned on the multilingual dataset generally outperformed XLM-RoBERTa fine-tuned on a single language. However, the results in Table 5.10 point to the fact that a model pre-trained on a single language (RoBERTa) outperforms a model pre-trained on multiple languages (XLM-RoBERTa). The dominance of the monolingual model is not surprising as a model pre-trained on a single language had a much smaller domain to learn than a multilingual model, and as such might have been able to learn linguistic and structural patterns that the multilingual model failed to capture. So, for the languages where a pre-trained model does exist (for example, BERT for English, CamemBERT (Müller et al., 2020) for French, HerBERT (Rybak et al., 2020) for Polish), that model may outperform the multilingual. However, it is unrealistic to assume that there can be a pre-trained model for each of the 200+ languages of Wikipedia. For the languages for which there is no pre-trained model, then the multilingual model becomes essential. As we saw in Sections 5.3 and 5.3.1 the multilingual model is capable of transferring knowledge to unseen languages, which proves its potential for low-resource languages for which a full pre-trained model is not realistic.

Conclusions and Future Work

5.5 Conclusions

In this thesis, we introduced the novel task of entity insertion. We conceptually justified the novelty of the task of entity insertion and empirically proved that there is a large benefit to be extracted from this task, by showing that in more than 55% of the cases, none of the existing text is suitable to insert the entity, and new text has to be inserted along with the inserted entity. To enable future research into entity insertion, we produced a dataset of links in 110 languages extracted from Wikipedia and used a subset of 21 languages to train and evaluate an entity insertion model.

We found that multilingual encoder models are capable of collating the knowledge acquired from each language to improve performance over all languages, consistently beating monolingual models trained on each downstream language. Furthermore, the model is capable of transferring knowledge to languages unseen during fine-tuning, in a zero-shot manner, while retaining from 85% to 98% performance. We additionally introduced several modifications to the base modelling framework: (1) a novel data augmentation strategy, (2) knowledge injection, and (3) a two-stage training pipeline for domain adaptation. We found that, for small languages, the two-stage training pipeline is vital for model performance but, for large languages, the first stage yields little benefit. The remaining modifications have a small impact on performance, albeit more significant for smaller languages. Finally, we showed that the starting pre-trained model has a large impact on performance, but model size does not seem to benefit entity insertion in the monolingual scenario.

The results from this thesis are a starting point for the novel task of entity insertion and show that multilingual entity insertion in Wikipedia is a realistic and useful task.

5.6 Future work

Our work leaves open multiple avenues for future work.

First, it would be relevant to compare our model's performance with newer and larger LLMs such as GPT-3/4 (Brown et al., 2020; OpenAI et al., 2023), acting as a very strong baseline. We

saw that our model vastly outperformed the classical baselines, but a comparison with other learning-based approaches is warranted.

Additionally, even though we didn't see a boost in performance from larger models in the English language, the multilingual case is much more complex, requiring the model to understand interactions and patterns in a much broader domain. As such, the additional problem complexity might warrant the need for larger models, leading to a further boost in performance. Likewise, even though RoBERTa outperformed BERT and T5 by a large margin, we saw there is a large drop between RoBERTa and XLM-RoBERTa, which invites further exploration of multilingual models such as mT5 and multilingual BERT.

Finally, we can try to explore additional signals present in the data, such as local Wikipedia graph structure knowledge or additional information regarding the candidate context. A very strong signal is the links already existing in the candidate context, as these indicate entities related to the context. Giving additional information to the model about such entities might help produce a better representation of the candidate, and thus produce better relevance embeddings.

Bibliography

Arora, A., Garcia-Duran, A., and West, R. (2021). Low-Rank Subspaces for Unsupervised Entity Linking. In *EMNLP*, pages 8037–8054.

Arora, A., West, R., and Gerlach, M. (2023). Orphan Articles: The Dark Matter of Wikipedia. *CoRR*, abs/2306.03940.

Bhagavatula, C., Feldman, S., Power, R., and Ammar, W. (2018). Content-Based Citation Recommendation. In Walker, M. A., Ji, H., and Stent, A., editors, *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2018, New Orleans, Louisiana, USA, June 1-6, 2018, Volume 1 (Long Papers)*, pages 238–251. Association for Computational Linguistics.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language Models are Few-Shot Learners. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H., editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.

Brutzkus, A. and Globerson, A. (2019). Why do Larger Models Generalize Better? A Theoretical Perspective via the XOR Problem. In Chaudhuri, K. and Salakhutdinov, R., editors, *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pages 822–830. PMLR.

Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., and Stoyanov, V. (2020). Unsupervised Cross-lingual Representation Learning at Scale. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J. R., editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, pages 8440–8451. Association for Computational Linguistics.

Čuljak, M., Spitz, A., West, R., and Arora, A. (2022). Strong Heuristics for Named Entity Linking. In *NAACL-SRW*, pages 235–246.

Devlin, J., Chang, M., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Burstein, J., Doran, C., and Solorio, T., editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pages 4171–4186. Association for Computational Linguistics.

Du, C., Popat, K., Martin, L., and Petroni, F. (2022). Entity Tagging: Extracting Entities in Text Without Mention Supervision. *CoRR*, abs/2209.06148.

Finkel, J. R., Grenager, T., and Manning, C. D. (2005). Incorporating non-local information into information extraction systems by gibbs sampling. In Knight, K., Ng, H. T., and Oflazer, K., editors, *ACL 2005, 43rd Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference, 25-30 June 2005, University of Michigan, USA*, pages 363–370. The Association for Computer Linguistics.

Fu, X., Shi, W., Zhao, Z., Yu, X., and Roth, D. (2020). Design Challenges for Low-resource Cross-lingual Entity Linking. *CoRR*, abs/2005.00692.

Gao, L., Dai, Z., and Callan, J. (2021). Rethink Training of BERT Rerankers in Multi-stage Retrieval Pipeline. In Hiemstra, D., Moens, M., Mothe, J., Perego, R., Potthast, M., and Sebastiani, F., editors, *Advances in Information Retrieval - 43rd European Conference on IR Research, ECIR 2021, Virtual Event, March 28 - April 1, 2021, Proceedings, Part II*, volume 12657 of *Lecture Notes in Computer Science*, pages 280–286. Springer.

Gerlach, M., Miller, M., Ho, R., Harlan, K., and Difallah, D. E. (2021). Multilingual Entity Linking System for Wikipedia with a Machine-in-the-Loop Approach. In Demartini, G., Zuccon, G., Culpepper, J. S., Huang, Z., and Tong, H., editors, *CIKM '21: The 30th ACM International Conference on Information and Knowledge Management, Virtual Event, Queensland, Australia, November 1 - 5, 2021*, pages 3818–3827. ACM.

Gururangan, S., Marasovic, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D., and Smith, N. A. (2020). Don't Stop Pretraining: Adapt Language Models to Domains and Tasks. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J. R., editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, pages 8342–8360. Association for Computational Linguistics.

Han, S., Wang, X., Bendersky, M., and Najork, M. (2020). Learning-to-Rank with BERT in TF-Ranking. *CoRR*, abs/2004.08476.

Hoffart, J., Yosef, M. A., Bordino, I., Fürstenau, H., Pinkal, M., Spaniol, M., Taneva, B., Thater, S., and Weikum, G. (2011). Robust Disambiguation of Named Entities in Text. In *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing, EMNLP 2011, 27-31 July 2011, John McIntyre Conference Centre, Edinburgh, UK, A meeting of SIGDAT, a Special Interest Group of the ACL*, pages 782–792. ACL.

Huang, D. W. C., Geva, S., and Trotman, A. (2008). Overview of the INEX 2008 Link the Wiki Track. In Geva, S., Kamps, J., and Trotman, A., editors, *Advances in Focused Retrieval, 7th International Workshop of the Initiative for the Evaluation of XML Retrieval, INEX 2008, Dagstuhl Castle, Germany, December 15-18, 2008. Revised and Selected Papers*, volume 5631 of *Lecture Notes in Computer Science*, pages 314–325. Springer.

Jiang, J., Zhang, M., Li, C., Bendersky, M., Golbandi, N., and Najork, M. (2019). Semantic Text Matching for Long-Form Documents. In Liu, L., White, R. W., Mantrach, A., Silvestri, F., McAuley, J. J., Baeza-Yates, R., and Zia, L., editors, *The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13-17, 2019*, pages 795–806. ACM.

Ju, J., Yang, J., and Wang, C. (2021). Text-to-Text Multi-view Learning for Passage Re-ranking. In Diaz, F., Shah, C., Suel, T., Castells, P., Jones, R., and Sakai, T., editors, *SIGIR '21: The 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, Virtual Event, Canada, July 11-15, 2021*, pages 1803–1807. ACM.

Lin, D. and Wu, X. (2009). Phrase Clustering for Discriminative Learning. In Su, K., Su, J., and Wiebe, J., editors, *ACL 2009, Proceedings of the 47th Annual Meeting of the Association for Computational Linguistics and the 4th International Joint Conference on Natural Language Processing of the AFNLP, 2-7 August 2009, Singapore*, pages 1030–1038. The Association for Computer Linguistics.

Lin, J., Nogueira, R. F., and Yates, A. (2021). *Pretrained Transformers for Text Ranking: BERT and Beyond*. Synthesis Lectures on Human Language Technologies. Morgan & Claypool Publishers.

Liu, N. F., Lee, K., and Toutanova, K. (2023). Anchor Prediction: Automatic Refinement of Internet Links. *CoRR*, abs/2305.14337.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. *CoRR*, abs/1907.11692.

Manning, C. D., Raghavan, P., and Schutze, H. (2012). *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, England.

Mihalcea, R. and Csomai, A. (2007). Wikify!: Linking Documents to Encyclopedic Knowledge. In Silva, M. J., Laender, A. H. F., Baeza-Yates, R. A., McGuinness, D. L., Olstad, B., Olsen, Ø. H., and Falcão, A. O., editors, *Proceedings of the Sixteenth ACM Conference on Information and Knowledge Management, CIKM 2007, Lisbon, Portugal, November 6-10, 2007*, pages 233–242. ACM.

Milne, D. N. and Witten, I. H. (2008). Learning to Link with Wikipedia. In Shanahan, J. G., Amer-Yahia, S., Manolescu, I., Zhang, Y., Evans, D. A., Kolcz, A., Choi, K., and Chowdhury, A., editors, *Proceedings of the 17th ACM Conference on Information and Knowledge Management, CIKM 2008, Napa Valley, California, USA, October 26-30, 2008*, pages 509–518. ACM.

Müller, L. M. B., Suárez, P. J. O., Dupont, Y., Romary, L., de la Clergerie, É., Seddah, D., and Sagot, B. (2020). CamemBERT: a Tasty French Language Model. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J. R., editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, pages 7203–7219. Association for Computational Linguistics.

Nogueira, R. F., Jiang, Z., Pradeep, R., and Lin, J. (2020). Document Ranking with a Pretrained Sequence-to-Sequence Model. In Cohn, T., He, Y., and Liu, Y., editors, *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020*, volume EMNLP 2020 of *Findings of ACL*, pages 708–718. Association for Computational Linguistics.

Nogueira, R. F., Yang, W., Cho, K., and Lin, J. (2019). Multi-Stage Document Ranking with BERT. *CoRR*, abs/1910.14424.

Nothman, J., Ringland, N., Radford, W., Murphy, T., and Curran, J. R. (2013). Learning multilingual named entity recognition from Wikipedia. *Artif. Intell.*, 194:151–175.

OpenAI, :, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., Bello, I., Berdine, J., Bernadett-Shapiro, G., Berner, C., Bogdonoff, L., Boiko, O., Boyd, M., Brakman, A.-L., Brockman, G., Brooks, T., Brundage, M., Button, K., Cai, T., Campbell, R., Cann, A., Carey, B., Carlson, C., Carmichael, R., Chan, B., Chang, C., Chantzis, F., Chen, D., Chen, S., Chen, R., Chen, J., Chen, M., Chess, B., Cho, C., Chu, C., Chung, H. W., Cummings, D., Currier, J., Dai, Y., Decareaux, C., Degry, T., Deutsch, N., Deville, D., Dhar, A., Dohan, D., Dowling, S., Dunning, S., Ecoffet, A., Eleti, A., Eloundou, T., Farhi, D., Fedus, L., Felix, N., Fishman, S. P., Forte, J., Fulford, I., Gao, L., Georges, E., Gibson, C., Goel, V., Gogineni, T., Goh, G., Gontijo-Lopes, R., Gordon, J., Grafstein, M., Gray, S., Greene, R., Gross, J., Gu, S. S., Guo, Y., Hallacy, C., Han, J., Harris, J., He, Y., Heaton, M., Heidecke, J., Hesse, C., Hickey, A., Hickey, W., Hoeschele, P., Houghton, B., Hsu, K., Hu, S., Hu, X., Huizinga, J., Jain, S., Jain, S., Jang, J., Jiang, A., Jiang, R., Jin, H., Jin, D., Jomoto, S., Jonn, B., Jun, H., Kaftan, T., Łukasz Kaiser, Kamali, A., Kanitscheider, I., Keskar, N. S., Khan, T., Kilpatrick, L., Kim, J. W., Kim, C., Kim, Y., Kirchner, H., Kiros, J., Knight, M., Kokotajlo, D., Łukasz Kondraciuk, Kondrich, A., Konstantinidis, A., Kosic, K., Krueger, G., Kuo, V., Lampe, M., Lan, I., Lee, T., Leike, J., Leung, J., Levy, D., Li, C. M., Lim, R., Lin, M., Lin, S., Litwin, M., Lopez, T., Lowe, R., Lue, P., Makanju, A., Malfacini, K., Manning, S., Markov, T., Markovski, Y., Martin, B., Mayer, K., Mayne, A., McGrew, B., McKinney, S. M., McLeavey, C., McMillan, P., McNeil, J., Medina, D., Mehta, A., Menick, J., Metz, L., Mishchenko, A., Mishkin, P., Monaco, V., Morikawa, E., Mossing, D., Mu, T., Murati, M., Murk, O., Mély, D., Nair, A., Nakano, R., Nayak, R., Neelakantan, A., Ngo, R., Noh, H., Ouyang, L., O'Keefe, C., Pachocki, J., Paino, A., Palermo, J., Pantuliano, A., Parascandolo, G., Parish, J., Parparita, E., Passos, A., Pavlov, M., Peng, A., Perelman, A., de Avila Belbute Peres, F., Petrov, M., de Oliveira Pinto, H. P., Michael, Pokorny, Pokrass, M., Pong, V., Powell, T., Power, A., Power, B., Proehl, E., Puri, R., Radford, A., Rae, J., Ramesh, A., Raymond, C., Real, F., Rimbach, K., Ross, C., Rotsted,

B., Roussez, H., Ryder, N., Saltarelli, M., Sanders, T., Santurkar, S., Sastry, G., Schmidt, H., Schnurr, D., Schulman, J., Selsam, D., Sheppard, K., Sherbakov, T., Shieh, J., Shoker, S., Shyam, P., Sidor, S., Sigler, E., Simens, M., Sitkin, J., Slama, K., Sohl, I., Sokolowsky, B., Song, Y., Staudacher, N., Such, F. P., Summers, N., Sutskever, I., Tang, J., Tezak, N., Thompson, M., Tillet, P., Tootoonchian, A., Tseng, E., Tuggle, P., Turley, N., Tworek, J., Uribe, J. F. C., Vallone, A., Vijayvergiya, A., Voss, C., Wainwright, C., Wang, J. J., Wang, A., Wang, B., Ward, J., Wei, J., Weinmann, C., Welihinda, A., Welinder, P., Weng, J., Weng, L., Wiethoff, M., Willner, D., Winter, C., Wolrich, S., Wong, H., Workman, L., Wu, S., Wu, J., Wu, M., Xiao, K., Xu, T., Yoo, S., Yu, K., Yuan, Q., Zaremba, W., Zellers, R., Zhang, C., Zhang, M., Zhao, S., Zheng, T., Zhuang, J., Zhuk, W., and Zoph, B. (2023). GPT-4 Technical Report.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *J. Mach. Learn. Res.*, 21:140:1–140:67.

Robertson, S. E. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Found. Trends Inf. Retr.*, 3(4):333–389.

Rybak, P., Mroczkowski, R., Tracz, J., and Gawlik, I. (2020). KLEJ: Comprehensive Benchmark for Polish Language Understanding. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J. R., editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, pages 1191–1201. Association for Computational Linguistics.

Simon, J. B., Karkada, D., Ghosh, N., and Belkin, M. (2023). More is Better in Modern Machine Learning: when Infinite Overparameterization is Optimal and Overfitting is Obligatory. *CoRR*, abs/2311.14646.

Soltanolkotabi, M., Javanmard, A., and Lee, J. D. (2019). Theoretical Insights Into the Optimization Landscape of Over-Parameterized Shallow Neural Networks. *IEEE Trans. Inf. Theory*, 65(2):742–769.

Taylor, R., Kardas, M., Cucurull, G., Scialom, T., Hartshorn, A., Saravia, E., Poulton, A., Kerkez, V., and Stojnic, R. (2022). Galactica: A Large Language Model for Science. *CoRR*, abs/2211.09085.

Trotman, A., Puurula, A., and Burgess, B. (2014). Improvements to BM25 and Language Models Examined. In Culpepper, J. S., Park, L. A. F., and Zuccon, G., editors, *Proceedings of the 2014 Australasian Document Computing Symposium, ADCS 2014, Melbourne, VIC, Australia, November 27-28, 2014*, page 58. ACM.

van Hulst, J. M., Hasibi, F., Dercksen, K., Balog, K., and de Vries, A. P. (2020). REL: An Entity Linker Standing on the Shoulders of Giants. In Huang, J. X., Chang, Y., Cheng, X., Kamps, J., Murdock, V., Wen, J., and Liu, Y., editors, *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020*, pages 2197–2200. ACM.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is All you Need. In Guyon, I., von Luxburg, U., Bengio, S., Wallach, H. M., Fergus, R., Vishwanathan, S. V. N., and Garnett, R., editors, *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5998–6008.

West, R., Precup, D., and Pineau, J. (2009). Completing Wikipedia's Hyperlink Structure through Dimensionality Reduction. In Cheung, D. W., Song, I., Chu, W. W., Hu, X., and Lin, J., editors, *Proceedings of the 18th ACM Conference on Information and Knowledge Management, CIKM 2009, Hong Kong, China, November 2-6, 2009*, pages 1097–1106. ACM.

Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A., Barua, A., and Raffel, C. (2021). mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer. In Toutanova, K., Rumshisky, A., Zettlemoyer, L., Hakkani-Tür, D., Beltagy, I., Bethard, S., Cotterell, R., Chakraborty, T., and Zhou, Y., editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2021, Online, June 6-11, 2021*, pages 483–498. Association for Computational Linguistics.

Zhang, W., Hua, W., and Stratos, K. (2022). EntQA: Entity Linking as Question Answering. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net.

Zhuang, H., Qin, Z., Jagerman, R., Hui, K., Ma, J., Lu, J., Ni, J., Wang, X., and Bendersky, M. (2023). RankT5: Fine-Tuning T5 for Text Ranking with Ranking Losses. In Chen, H., Duh, W. E., Huang, H., Kato, M. P., Mothe, J., and Poblete, B., editors, *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2023, Taipei, Taiwan, July 23-27, 2023*, pages 2308–2313. ACM.

A Full Link Insertion Statistics

Similarly to the plot of Figure 1 we computed the number of links in each link insertion category for all the link insertion strategies for all the considered languages. Table A.1 shows the relative and absolute number of links in each category, and Table A.2 shows the aggregate statistics across all 110 languages. The final results are consistent with the results in Figure 1 and they prove that most links are added at the same time as the text to be linked, proving the need for powerful entity insertion models.

Appendix A. Full Link Insertion Statistics

Table A.1: **Full statistics for added links for all 110 processed languages.** Results describe links added from 2023-10-01 until 2023-11-01. For English (en), only a sample of the links was extracted due to time and computational limits. Categories are as follows: (1): `text_present`, (2): `missing_mention`, (3): `missing_sentence`, (4): `missing_span`, (5): `missing_section`.

	(1)	(2)	(3)	(4)	(5)	Total		(1)	(2)	(3)	(4)	(5)	Total
en*	41492 (23.9%)	33679 (19.4%)	32583 (18.7%)	48325 (27.8%)	17808 (10.2%)	173887	de	24331 (25.8%)	15327 (16.2%)	17937 (19.0%)	27367 (29.0%)	9363 (9.9%)	94325
ja	22607 (28.6%)	21860 (27.7%)	17935 (22.7%)	12282 (15.5%)	4374 (5.5%)	79058	es	20113 (30.3%)	19183 (28.9%)	5899 (8.9%)	13763 (20.7%)	7419 (11.2%)	66377
fr	19626 (30.4%)	11290 (17.5%)	10479 (16.2%)	18293 (28.3%)	4851 (7.5%)	64539	it	12783 (28.0%)	7754 (17.0%)	7516 (16.5%)	11158 (24.4%)	6430 (14.1%)	45641
he	13190 (35.9%)	5837 (15.9%)	5087 (13.8%)	9014 (24.5%)	3653 (9.9%)	36781	ru	11694 (34.5%)	5406 (16.0%)	4627 (13.7%)	9266 (27.4%)	2860 (8.4%)	33853
zh	8630 (30.6%)	7095 (25.2%)	5329 (18.9%)	5531 (19.6%)	1624 (5.8%)	28209	pl	7488 (27.4%)	3956 (14.5%)	4455 (16.3%)	7525 (27.6%)	3873 (14.2%)	27297
pt	6677 (27.5%)	3728 (15.4%)	2477 (10.2%)	7109 (29.3%)	4279 (17.6%)	24270	nl	6021 (25.4%)	5130 (21.6%)	4085 (17.2%)	5465 (23.1%)	2996 (12.6%)	23697
ko	5964 (27.9%)	3509 (16.4%)	4009 (18.7%)	5425 (25.4%)	2491 (11.6%)	21398	ca	4506 (24.5%)	2425 (13.2%)	1518 (8.2%)	6783 (36.8%)	3178 (17.3%)	18410
fa	7615 (42.1%)	3447 (19.0%)	1347 (7.4%)	3571 (19.7%)	2127 (11.7%)	18107	ar	7229 (40.5%)	2661 (14.9%)	1670 (9.4%)	3996 (22.4%)	2304 (12.9%)	17860
tr	4474 (29.9%)	2427 (16.2%)	1652 (11.0%)	3983 (26.6%)	2449 (16.3%)	14985	fi	5187 (37.8%)	2162 (15.7%)	1924 (14.0%)	2907 (21.2%)	1548 (11.3%)	13728
id	3267 (23.8%)	2216 (16.1%)	1749 (12.7%)	4609 (33.6%)	1886 (13.7%)	13727	cs	3687 (30.0%)	1990 (16.2%)	1556 (12.6%)	3201 (26.0%)	1870 (15.2%)	12304
vi	3797 (31.7%)	1562 (13.0%)	999 (8.3%)	4117 (34.4%)	1510 (12.6%)	11985	sv	3453 (32.2%)	1978 (18.4%)	1391 (13.0%)	2610 (24.3%)	1297 (12.1%)	10729
hu	2422 (31.0%)	1193 (15.3%)	1013 (13.0%)	2467 (31.6%)	708 (9.1%)	7803	no	1673 (23.0%)	1053 (14.5%)	1048 (14.4%)	2543 (34.9%)	966 (13.3%)	7283
el	3032 (45.0%)	824 (12.2%)	445 (6.6%)	1937 (28.7%)	505 (7.5%)	6743	az	1982 (31.4%)	703 (11.1%)	666 (10.5%)	2586 (41.0%)	376 (6.0%)	6313
eo	1218 (20.9%)	2237 (38.3%)	576 (9.9%)	1045 (17.9%)	762 (13.1%)	5838	sr	2251 (41.4%)	735 (13.5%)	514 (9.5%)	1310 (24.1%)	627 (11.5%)	5437
ur	3941 (74.8%)	666 (12.6%)	157 (3.0%)	200 (3.8%)	304 (5.8%)	5268	eu	1047 (20.4%)	434 (8.4%)	1515 (29.5%)	1543 (30.0%)	603 (11.7%)	5142
bg	1546 (32.2%)	752 (15.6%)	400 (8.3%)	1578 (32.8%)	531 (11.0%)	4807	et	1172 (24.9%)	857 (18.2%)	813 (17.3%)	1201 (25.5%)	662 (14.1%)	4705
uz	2631 (60.7%)	318 (7.3%)	172 (4.0%)	527 (12.1%)	690 (15.9%)	4338	ro	1103 (25.7%)	546 (12.7%)	779 (18.2%)	1364 (31.8%)	495 (11.5%)	4287
gl	1030 (24.2%)	461 (10.8%)	307 (7.2%)	1847 (43.4%)	613 (14.4%)	4258	simple	1344 (35.1%)	621 (16.2%)	600 (15.7%)	734 (19.2%)	532 (13.9%)	3831
hy	1354 (35.6%)	1007 (26.5%)	387 (10.2%)	625 (16.5%)	426 (11.2%)	3799	hr	1107 (32.1%)	531 (15.4%)	220 (6.4%)	719 (20.8%)	876 (25.4%)	3453
sk	1028 (32.9%)	497 (15.9%)	384 (12.3%)	782 (25.0%)	432 (13.8%)	3123	be	839 (27.1%)	386 (12.5%)	554 (17.9%)	876 (28.3%)	437 (14.1%)	3092
ms	897 (32.7%)	710 (25.9%)	298 (10.9%)	414 (15.1%)	427 (15.5%)	2746	lt	497 (20.6%)	372 (15.4%)	450 (18.6%)	606 (25.1%)	488 (20.2%)	2413
da	679 (28.7%)	369 (15.6%)	298 (12.6%)	619 (26.2%)	398 (16.8%)	2363	hi	1002 (42.8%)	721 (30.8%)	84 (3.6%)	156 (6.7%)	379 (16.2%)	2342
lv	541 (27.7%)	328 (16.8%)	229 (11.7%)	500 (25.6%)	358 (18.3%)	1956	oc	552 (28.9%)	210 (11.0%)	164 (8.6%)	857 (44.8%)	129 (6.7%)	1912
ka	1106 (61.9%)	233 (13.0%)	87 (4.9%)	222 (12.4%)	138 (7.7%)	1786	kk	942 (59.1%)	309 (19.4%)	85 (5.3%)	140 (8.8%)	119 (7.5%)	1595
sl	409 (26.5%)	273 (17.7%)	123 (8.0%)	494 (32.0%)	245 (15.9%)	1544	af	695 (48.0%)	104 (7.2%)	161 (11.1%)	262 (18.1%)	227 (15.7%)	1449
la	357 (29.0%)	266 (21.6%)	220 (17.9%)	315 (25.6%)	71 (5.8%)	1229	mg	406 (33.7%)	164 (13.6%)	55 (4.6%)	157 (13.1%)	421 (35.0%)	1203
te	260 (22.4%)	158 (13.6%)	95 (8.2%)	314 (27.1%)	333 (28.7%)	1160	ta	362 (31.6%)	205 (17.9%)	67 (5.8%)	315 (27.5%)	197 (17.2%)	1146
is	158 (15.1%)	75 (7.2%)	144 (13.8%)	559 (53.5%)	109 (10.4%)	1045	bs	352 (36.3%)	54 (5.6%)	144 (14.9%)	257 (26.5%)	162 (16.7%)	969
mk	331 (35.9%)	157 (17.0%)	120 (13.0%)	202 (21.9%)	113 (12.2%)	923	br	222 (26.1%)	197 (23.1%)	116 (13.6%)	198 (23.2%)	119 (14.0%)	852
sh	217 (26.9%)	107 (13.3%)	94 (11.6%)	279 (34.6%)	110 (13.6%)	807	lb	184 (24.4%)	148 (19.6%)	168 (22.3%)	117 (15.5%)	137 (18.2%)	754
ba	212 (32.7%)	60 (9.2%)	21 (3.2%)	206 (31.7%)	150 (23.1%)	649	ku	263 (42.8%)	130 (21.2%)	31 (5.0%)	163 (26.5%)	27 (4.4%)	614
fy	187 (32.3%)	83 (14.3%)	50 (8.6%)	183 (31.6%)	76 (13.1%)	579	sw	173 (31.0%)	200 (35.8%)	100 (17.9%)	54 (9.7%)	31 (5.6%)	558
sq	128 (24.5%)	77 (14.7%)	85 (16.3%)	131 (25.0%)	102 (19.5%)	523	kn	315 (61.3%)	27 (5.3%)	15 (2.9%)	83 (16.1%)	74 (14.4%)	514
tl	115 (22.5%)	114 (22.3%)	18 (3.5%)	144 (28.1%)	121 (23.6%)	512	mn	250 (53.5%)	68 (14.6%)	26 (5.6%)	81 (17.3%)	42 (9.0%)	467
as	214 (46.6%)	77 (16.8%)	21 (4.6%)	54 (11.8%)	93 (20.3%)	459	cy	224 (58.0%)	56 (14.5%)	41 (10.6%)	44 (11.4%)	21 (5.4%)	386
cv	72 (23.7%)	67 (22.0%)	51 (16.8%)	74 (24.3%)	40 (13.2%)	304	ha	201 (76.7%)	11 (4.2%)	4 (1.5%)	17 (6.5%)	29 (11.1%)	262
ne	100 (40.0%)	94 (37.6%)	5 (2.0%)	10 (4.0%)	41 (16.4%)	250	io	14 (6.1%)	46 (20.0%)	65 (28.3%)	80 (34.8%)	25 (10.9%)	230
ga	61 (29.9%)	34 (16.7%)	18 (8.8%)	79 (38.7%)	12 (5.9%)	204	jv	66 (41.0%)	21 (13.0%)	3 (1.9%)	46 (28.6%)	25 (15.5%)	161
pa	23 (16.5%)	40 (28.8%)	39 (28.1%)	13 (9.4%)	24 (17.3%)	139	ary	43 (33.3%)	29 (22.5%)	14 (10.9%)	32 (24.8%)	11 (8.5%)	129
tg	51 (41.5%)	38 (30.9%)	6 (4.9%)	21 (17.1%)	7 (5.7%)	123	km	62 (65.3%)	22 (23.2%)	3 (3.2%)	4 (4.2%)	4 (4.2%)	95
tt	17 (18.1%)	13 (13.8%)	35 (37.2%)	9 (9.6%)	20 (21.3%)	94	an	8 (11.4%)	6 (8.6%)	13 (18.6%)	18 (25.7%)	25 (35.7%)	70
mr	46 (68.7%)	10 (14.9%)	6 (9.0%)	4 (6.0%)	1 (1.5%)	67	so	29 (48.3%)	10 (16.7%)	3 (5.0%)	17 (28.3%)	1 (1.7%)	60
ast	14 (28.6%)	22 (44.9%)	1 (2.0%)	11 (22.4%)	1 (2.0%)	49	ce	0 (0.0%)	2 (4.2%)	0 (0.0%)	45 (93.8%)	1 (2.1%)	48
sco	18 (45.0%)	15 (37.5%)	3 (7.5%)	2 (5.0%)	2 (5.0%)	40	si	9 (25.0%)	3 (8.3%)	3 (8.3%)	16 (44.4%)	5 (13.9%)	36
gu	18 (62.1%)	5 (17.2%)	0 (0.0%)	4 (13.8%)	2 (6.9%)	29	lmo	3 (11.5%)	2 (7.7%)	1 (3.8%)	1 (3.8%)	19 (73.1%)	26
azb	15 (68.2%)	3 (13.6%)	4 (18.2%)	0 (0.0%)	0 (0.0%)	22	bar	9 (42.9%)	2 (9.5%)	2 (9.5%)	7 (33.3%)	1 (4.8%)	21
min	11 (52.4%)	2 (9.5%)	0 (0.0%)	1 (4.8%)	7 (33.3%)	21	yi	12 (57.1%)	2 (9.5%)	1 (4.8%)	6 (28.6%)	0 (0.0%)	21
om	13 (72.2%)	0 (0.0%)	1 (5.6%)	3 (16.7%)	1 (5.6%)	18	sa	4 (22.2%)	5 (27.8%)	0 (0.0%)	0 (0.0%)	9 (50.0%)	18
gd	4 (28.6%)	2 (14.3%)	5 (35.7%)	3 (21.4%)	0 (0.0%)	14	sd	8 (57.1%)	1 (7.1%)	3 (21.4%)	2 (14.3%)	0 (0.0%)	14
am	9 (75.0%)	3 (25.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	12	lo	4 (36.4%)	3 (27.3%)	3 (27.3%)	1 (9.1%)	0 (0.0%)	11
ps	5 (50.0%)	5 (50.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	10	vo	0 (0.0%)	7 (100.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	7
su	3 (50.0%)	0 (0.0%)	1 (16.7%)	2 (33.3%)	0 (0.0%)	6	scn	0 (0.0%)	3 (60.0%)	2 (40.0%)	0 (0.0%)	0 (0.0%)	5
ug	0 (0.0%)	1 (100.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	1	xh	1 (100.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	1
nds-nl	0	0	0	0	0	0	pms	0	0	0	0	0	0
uk	0	0	0	0	0	0	war	0	0	0	0	0	0

Table A.2: **Aggregated statistics for added links for all 110 processed languages.**

	# Links	Micro Average Distribution	Macro Average Distribution
`text_present`	287754	29.4%	35.8%
`missing_mention`	185024	18.9%	19.0%
`missing_sentence`	150679	15.4%	11.2%
`missing_span`	248838	25.4%	21.8%
`missing_section`	106465	10.9%	12.2%
Total	978760	–	–

B Single Encoder vs Triple Encoder

In early iterations of our work, we explored a different model architecture, represented in Figure B.1. This architecture used the additional knowledge of the source article. Given the amount of text that needed to be encoded, and considering that most transformers have a limited number of tokens they can process, we chose to encode each of the three components separately. We had the following input representations:

- Source Article: [CLS]<Source Title>[SEP]<Source Lead>
- Candidate: [CLS]<Source Section>[SEP]<Target Mention>[SEP]<Context>
- Target Title: [CLS]<Target Title>[SEP]<Target Lead>

Each of the components of the triplet was encoded independently, and then stacked together. Finally, an MLP captured the interactions between the three embeddings and produced a relevance score.

The key intuition behind this architecture was to represent a link as a knowledge triplet `<src, text, tgt>`, and the overall architecture was supposed to predict whether the triplet was correct. However, we found that such an architecture decayed into a state where the target and source embeddings were independent of the input, always producing the same embedding. We believe that the model relied exclusively on the semantic knowledge contained in the list of target mentions to identify whether the entity should be inserted in the candidate text span, and the source and article embeddings decayed into a global average optimum that maximised the performance of the MLP for the candidate embedding. Nevertheless, this meant that all the knowledge about the target entity contained in the target lead was being ignored.

To take advantage of the total available information, we moved to the architecture described in Chapter 4. We removed the source title and the source lead, driven by the token limit of the transformer architecture. We believed that this knowledge provided the least marginal gain

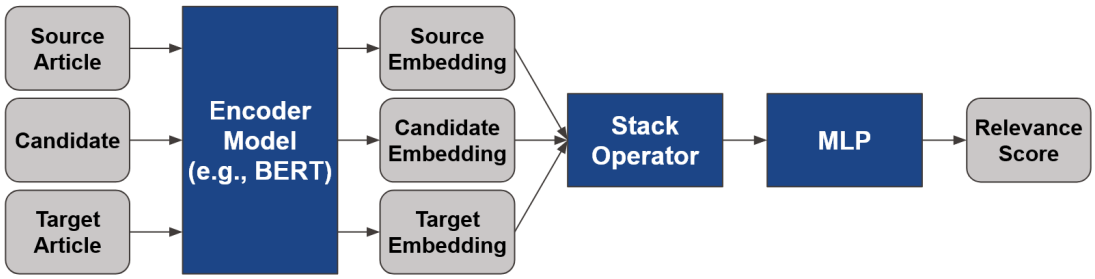


Figure B.1: **Discarded architecture diagram.**

from the three components of the triplet, at a cost of token space for the candidate and the target, as the source article knowledge only gave additional context to the candidate text span.

We additionally moved to a single encoder for two reasons. First, the transformer architecture is more expressive than an MLP, and as such it was better suited to capture the interactions between the candidate and the target. With only two knowledge sources instead of three, we felt we had sufficient token space for each source to capture enough semantic information for each input. Second, by relying on one single embedding, the embedding couldn't decay into a global average optimum which provided no information about the input, because the relevance score was entirely dependent on the representation power of that single embedding.